

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Ingeniería de Computadores

Identificación de voces e instrumentos en una obra musical

Martín Quevedo Poyal

Dirección

Basilio Sierra Araujo
Izaro Goienetxea Urkizu

Junio de 2026

Resumen

Este Trabajo de Fin de Grado consiste en un acercamiento al problema de separación de fuentes en una obra musical que persigue, dada una pieza musical, obtener mediante diferentes métodos los elementos sonoros de los que se compone.

Con este problema en mente se ha elegido el aprendizaje automático como método para llevar a cabo la tarea y se comparan diferentes métodos para valorar numéricamente el aprendizaje realizado por el sistema en una misma arquitectura. Se obtiene finalmente así una comparativa de aprendizaje, un sistema que permite obtener elementos sonoros de una pieza musical, y una interfaz gráfica que proporciona accesibilidad a un usuario promedio ajeno al tema.

ODS

Los Objetivos de Desarrollo Sostenible (*ODS*), establecidos por las Naciones Unidas, constituyen un marco global que busca abordar los grandes desafíos sociales, económicos y medioambientales que enfrenta la humanidad. La investigación y el desarrollo tecnológico, especialmente en campos como la inteligencia artificial y el aprendizaje automático, pueden desempeñar un papel clave en la consecución de estos objetivos. Se definen de acuerdo al diagrama de la Figura 1:



Figura 1: Objetivos de desarrollo sostenible

Este proyecto , centrado en el desarrollo y entrenamiento de modelos de aprendizaje automático con un objetivo concreto, siendo este la identificación y separación de elementos en una obra musical, se alinea con varios de los Objetivos de Desarrollo Sostenible establecidos por las Naciones Unidas. A continuación , se ponen en detalle los objetivos más relevantes con los que este proyecto presenta simpatía y se comenta la aportación de este a cada uno de ellos.

ODS 4: Educación de calidad

Este *ODS* tiene como objetivo garantizar una educación inclusiva, equitativa y de calidad y promover oportunidades de aprendizaje durante toda la

vida para todos.

El sistema presentado en este proyecto, promueve oportunidades de aprendizaje para todos, identificando las funcionalidades más importantes de un modelo de aprendizaje automático , ilustrando las aplicaciones de estos en un ejemplo del mundo real , como puede ser la identificación de elementos en una obra musical y finalmente presentandolas al usuario de forma interactiva y transversal , con explicaciones por parámetro y método que permiten tanto a usuarios experimentados como a primerizos la interacción e introducción a esta tecnología que cada día está mas presente en nuestras vidas.

ODS 3: Salud y bienestar

Este *ODS* busca garantizar una vida sana y promover el bienestar para todos en todas las edades.

Si bien es sabido que el baile (y el disfrute de la música) es una actividad que , como ya se ha demostrado científicamente por multitud de estudios, promueve una mejora de la salud física y emocional del individuo, la separación de fuentes de una obra es ampliamente utilizada para estos propósitos, bien por *DJs* , *Beatmakers* o *Samplers* mediante diferentes técnicas dentro de estos ámbitos. Si bien esto se hacía antiguamente con métodos más rudimentarios , la introducción de la inteligencia artificial y en concreto de las redes neuronales artificiales como herramienta en este tipo de métodos , facilita y mejora el proceso , lo que deriva en el resultado de una mejor herramienta para el artista y por ende en disfrute para el público de este.

Además, este tipo de herramientas son ampliamente utilizadas para la práctica *amateur* de instrumentos musicales , bien para obtener la pista musical de la guitarra en una canción y poder replicarla de oído en casa con tu propio instrumento , o bien para obtener la pista vocal y acompañarla con tu propia partitura.

Estas posibilidades que nos da la herramienta contribuyen lateralmente al bienestar emocional, abriendo bien un abanico de posibilidades creativas en casa o bien un abanico de mejoras en la metodología artística de los “artesanos” de estas disciplinas.

Índice general

Resumen	I
ODS	III
Índice general	V
Índice de figuras	VII
Índice de cuadros	IX
1 Introducción	1
2 Planificación	3
3 Herramientas utilizadas	9
4 Marco teórico	13
5 Estado del arte	31
6 Parte 1 : separación de fuentes	37
7 Parte 2 : interfaz	51
8 Integración del sistema	63
9 Evaluación del sistema	67
10 Conclusiones y trabajo futuro	75
11 Anexos	77
Bibliografía	81

Índice de figuras

1.	Objetivos de desarrollo sostenible	III
2.1.	Cronograma inicial del proyecto.	6
2.2.	Cronograma final del proyecto.	7
4.1.	Generación de una onda sonora longitudinal mediante el movimiento de un pistón al final de un tubo [1].	14
4.2.	Representación gráfica de las diferencias entre sonido mono y sonido estéreo.	16
4.3.	Proceso de computar una transformada en tiempo corto de Fourier a partir de una onda. Imagen de Bryan Pardo	17
4.4.	Hann function	18
4.5.	Diferentes generaciones de espectrogramas según el tratamiento de la STFT.	19
4.6.	Espectrograma estándar arriba y espectrograma de Mel abajo.	21
4.7.	Comparativa de clasificación entre aprendizaje supervisado y no supervisado. Imagen de Juan Domingo Farnos.	23
4.8.	Una sola capa de red neural artificial feedforward. Por Mcstrother - Trabajo propio, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=9515940	25
4.9.	Ilustración del proceso de descenso de gradiente.	26
6.1.	Ejemplo de una arquitectura RNN-LSTM.	38
6.2.	Average SNR and SDR by genre	47
7.1.	Barra lateral de configuración de la interfaz.	52
7.2.	Zona de carga de la GUI.	53
7.3.	Consola y logs de la GUI.	53
7.4.	Reproductores de audio de la interfaz.	54
7.5.	Ejemplo de representación gráfica de la fuente vocal.	55
7.6.	Botones principales de la interfaz.	56
7.7.	Botón de configuración.	56
7.8.	Parámetros de formación de transformadas de Fourier modificables en la interfaz visual.	56
7.9.	Parámetros intrínsecos del modelo modificables en la interfaz visual.	57

7.10. Parámetros modificables de entrenamiento de nuevos modelos en la interfaz visual.	57
7.11. Parámetros modificables de generación de efectos en la interfaz visual.	57
7.12. Parámetros modificables de generación de mezclas en la interfaz visual.	58
7.13. Entrada a la ventana de información sobre los parámetros de la configuración avanzada.	59
7.14. Ejemplo de una forma de onda mostrada en la interfaz al finalizar la separación de una pista.	60
7.15. Ejemplo de un espectrograma mostrado en la interfaz al finalizar la separación de una pista.	60
7.16. Ejemplo de <i>logs</i> mostrados en la consola de la interfaz.	61
7.17. Ejemplo de mensaje de error mostrado en la interfaz al intentar separar una pista sin haber cargado una configuración de modelo previa.	61
7.18. Ejemplo de indicador de espera en la interfaz mientras se está realizando el proceso de separar una pista.	62
7.19. Botón de la interfaz que descarga un archivo wav de fuente en el almacenamiento del usuario.	62
8.1. Diagrama de flujo para el backend.	64
9.1. Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de dos fuentes.	72
9.2. Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de cuatro fuentes.	72

Índice de cuadros

2.1. Comparación entre las horas estimadas y las horas reales dedicadas a los bloques y tareas del proyecto.	5
4.1. Pérdidas principales consideradas durante el entrenamiento.	28
4.2. Pérdidas avanzadas y experimentales consideradas durante el desarrollo.....	29
6.1. Configuración arquitectónica base empleada en los entrenamientos V2.....	40
9.1. Tabla para los resultados de separación en 2 fuentes	70
9.2. Tabla para los resultados de separación en 4 fuentes	70
9.3. Ranking para los resultados de separación en 2 fuentes	71
9.4. Ranking para los resultados de separación en 4 fuentes	71

CAPÍTULO 1 Introducción



La separación de fuentes musicales o sonoras es, a día de hoy, un campo abierto en el ámbito de la investigación del procesado de señal. Se puede considerar como fuente sonora toda aquella que genere sonido susceptible de ser captado por un sensor de audio, es decir, un micrófono. Por tanto, son fuentes sonoras los instrumentos musicales, la voz, o cualquier fuente de ruido natural o artificial.

En este trabajo se ha abordado el problema de la separación de fuentes en una obra musical utilizando las herramientas que nos proporcionan los avances tecnológicos de la última década. El problema en sí mismo, se atribuye al ámbito del procesamiento digital de señales y en él se plantea el siguiente reto: desarrollar métodos para que partiendo de una pieza musical, se obtengan los elementos sonoros que la componen, esto es, por ejemplo, obtener a partir de una pieza en formato de archivo digital del género cantautor donde confluyen una vocal y un instrumento, por ejemplo, una guitarra acústica, obtener un método que tras su aplicación nos proporcione por un lado una pista con la información sonora de la vocal, y por otro lado la de la guitarra acústica.

Los métodos que se desarrollan para dar solución a este problema tienen aplicaciones en distintos ámbitos relacionados con el procesamiento de audio. Entre ellas se encuentran la creación de versiones instrumentales o de karaoke, la remezcla de obras musicales, la restauración de grabaciones, la identificación y extracción de voces, el análisis automático de música y el apoyo a tareas de transcripción o clasificación sonora. También constituye un problema relevante para el estudio de técnicas de aprendizaje profundo aplicadas a señales temporales.

El problema que se trata en este trabajo presenta una dificultad considerable debido a que las fuentes pueden compartir regiones temporales y frecuenciales. Por ejemplo, una voz y un instrumento pueden producir energía en frecuencias similares o sonar simultáneamente y como consecuencia no existe una división directa de la pista que permita recuperar cada fuente sin intro-

1. INTRODUCCIÓN

ducir errores, interferencias o ruido de otros elementos no deseados.

A lo largo de este documento se abordan una introducción al ámbito del procesamiento digital de señales, del aprendizaje automático y de los conceptos fundamentales de ambos. Se describe también el análisis de las decisiones tomadas durante el proceso de diseño e implementación, los desafíos técnicos y el análisis de los resultados.

CAPÍTULO Planificación 2



El actual capítulo trata de recapitular la planificación seguida de acuerdo a los métodos comunes de planificación que se siguen en un proyecto de ingeniería. Para ello, se trazaron un alcance inicial del proyecto y unos objetivos a cumplir. Finalmente se comentan las incidencias surgidas, los cambios respecto a la planificación inicial, y el desarrollo por fases inicial y el que terminó siendo a lo largo del proyecto mediante un cronograma.

2.1. Alcance

Este trabajo tiene como alcance la exploración de modelos y distancias en un mismo sistema de aprendizaje automático orientado a la extracción de los elementos sonoros de una pieza musical: voz, bajo, batería y acompañamiento, junto con la implementación de una interfaz gráfica que permite hacer diferentes pruebas.

El trabajo abarca la implementación de distintas funciones de pérdida ($L1$, $L2$, pérdidas en frecuencia, pérdidas de máscara...), el entrenamiento controlado de modelos con una misma configuración de *hiperparámetros*, y la evaluación del rendimiento obtenido a través de métricas específicas.

El proyecto no pretende competir con modelos de última generación en separación musical, sino explorar el impacto de diferentes estrategias de entrenamiento y funciones de pérdida en arquitecturas más simples, trabajando con *datasets* ya creados y recursos de cómputo reducidos.

2.2. Objetivos

El principal objetivo del proyecto es el desarrollo de una herramienta que identifica y separa los diferentes elementos en una obra musical.

2. PLANIFICACIÓN

Para facilitar el llevar a cabo el objetivo principal, facilita la tarea dividirlo en batallas más pequeñas. Se pueden resumir en dos grupos a los que se suma un objetivo secundario:

- **1. Objetivo inicial:** separación de dos fuentes reduciendo la tarea a la identificación de únicamente vocales y acompañamiento.

Este objetivo se daría por logrado al conseguir una implementación que diese como salida dos archivos de audio donde se discerniesen dos elementos diferentes provenientes de la mezcla original, y que, en conjunto, sumasen esta. Este objetivo se daría por cumplido al tener un sistema implementado cuya salida consistiese en las dos fuentes de una pista con dos elementos con una distinción clara, sin presencia de interferencias, sin presencia de ruido y que en conjunto sumasen la pista original.

- **2. Objetivo a largo plazo:** una vez cumplido el objetivo inicial, el siguiente paso es llegar a la separación de cuatro fuentes recogiendo vocales, bajo, baterías y el resto del acompañamiento identificado como "*resto*".

Este objetivo se daría por cumplido al conseguir una implementación en código que diese como salida cuatro archivos de audio comprendiendo vocales, bajo, baterías y resto de elementos, con una distinción clara de los elementos, sin presencia de interferencias, sin presencia de ruido y que en conjunto sumasen la pista original.

Objetivo secundario: Adicionalmente, y, aunque no estaba dentro de los objetivos iniciales y se tomó más como una propuesta a realizar en caso de que lo permitiese la planificación, se planteó desarrollar una interfaz gráfica muy sencilla que permite cargar audios de prueba y obtener sus *stems*¹ separados, así como almacenar los resultados en forma de archivos de audio y gráficas con cierto enfoque didáctico.

En resumen, podríamos agrupar los objetivos en:

- **1. Objetivo principal:** llegar a un estado del proyecto donde se genere una salida que presente una diferencia respecto a la pista de entrada y que identifique al menos superficialmente el elemento objetivo.
- **2. Objetivos técnicos:** implementar un proceso funcional de entrenamiento para las diferentes funciones. Evaluar los resultados y proporcionar una comparativa de las funciones.
- **3. Objetivo secundario de la interfaz:** facilitar el uso, dar comprensión para principiantes en el campo y dar visualización del sistema.

¹ *STEMS* es el término anglosajón que refiere a las sub-pistas que componen una mezcla musical (en castellano, fuentes)

2.3. Cronograma

Inicialmente, la planificación del proyecto planteaba seguir una estructura de fases organizada según el cuadro 2.1.

Cuadro 2.1: Comparación entre las horas estimadas y las horas reales dedicadas a los bloques y tareas del proyecto.

Trabajo Fin de Grado	Horas estimadas	Horas reales
PT 1 Aprendizaje y desarrollo inicial	62	76
T 1.1 Análisis del problema e investigación técnica	24	22
T 1.2 Integración del modelo y de las funciones de pérdida	22	40
T 1.3 Diseño del preprocesamiento	16	14
PT 2 Entrenamiento	150	137
T 2.1 Diseño del protocolo de entrenamiento	10	10
T 2.2 Ajuste de los parámetros de entrenamiento	28	36
T 2.3 Entrenamiento de dos fuentes y análisis de resultados	22	40
T 2.4 Entrenamiento de cuatro fuentes y análisis de resultados	30	50
T 2.5 Organización de los resultados	10	8
PT 3 Implementación de la interfaz	44	39
T 3.1 Análisis de requisitos	8	6
T 3.2 Elección de herramientas	6	5
T 3.3 Diseño del flujo	14	12
T 3.4 Implementación	16	16
PT 4 Evaluación final	80	150
T 4.1 Definición del protocolo de evaluación	10	11
T 4.2 Preparación del conjunto de evaluación	12	12
T 4.3 Cálculo de métricas	4	4
T 4.4 Análisis y representación de los resultados	14	14
T 4.5 Elaboración de conclusiones y trabajo futuro	12	46
PT 5 Documentación	80	150
T 5.1 Redacción y revisión de la memoria final	18	58
T 5.2 Limpieza y documentación del código	4	6
TIEMPO TOTAL	300	400

La estimación global del proyecto asciende a aproximadamente 350 horas de trabajo efectivo. Esta cifra incluye las tareas de análisis, implementación, depuración, supervisión de entrenamientos, evaluación y documentación. No se contabiliza íntegramente el tiempo durante el cual los modelos se entrenan de forma automática, sino únicamente el esfuerzo dedicado a preparar, supervisar y analizar dichas ejecuciones. En la Figura 2.1 se puede ver el cronograma inicial

2. PLANIFICACIÓN

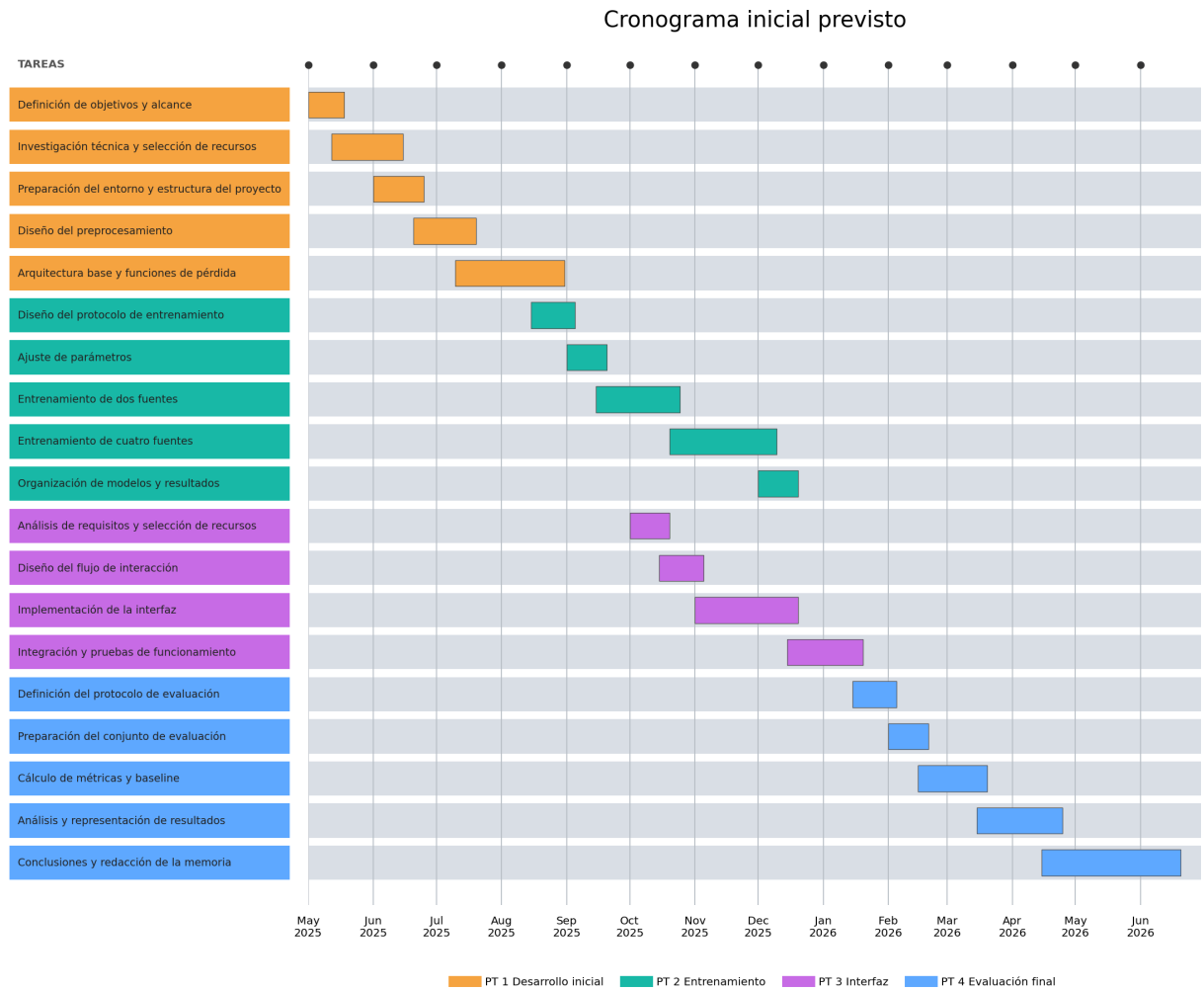


Figura 2.1: Cronograma inicial del proyecto.

2.4. Desviaciones respecto a la planificación inicial

Los cinco bloques principales en los que se divide la planificación y que recoge el cuadro 2.1 son los siguientes:

- Desarrollo inicial: recopilación de información e implementación inicial.
- Entrenamiento: implementación del proceso de entrenamiento de modelos,entrenamiento y recogida de resultados.
- Implementación de la interfaz.
- Evaluación final: análisis y comparación de resultados.

2.4. Desviaciones respecto a la planificación inicial

- Documentación: redacción de la memoria y de la pertinente documentación del código.

En esta planificación se asumía que, una vez entrenados los modelos, la evaluación consistiría principalmente en ejecutar las métricas definidas, probar los modelos generados y valorar los resultados obtenidos. Sin embargo, durante el desarrollo del proyecto fueron surgiendo varios problemas que modificaron el desarrollo previsto. Finalmente la ejecución del proyecto se puede ver en el cronograma de la Figura 2.1, las incidencias que han causado los retrasos en la planificación son las enumeradas a continuación.

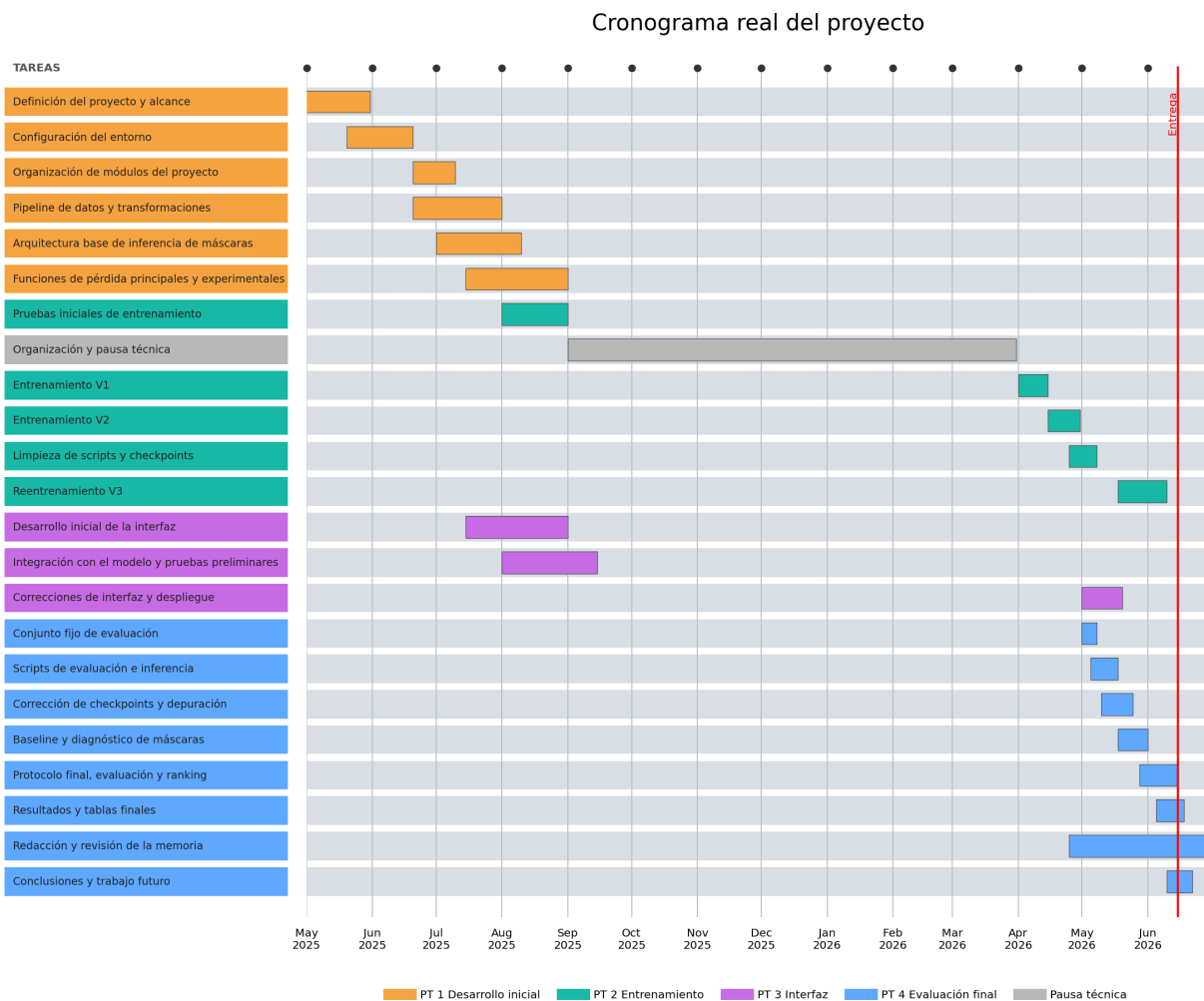


Figura 2.2: Cronograma final del proyecto.

Las incidencias que causaron este aumento en el tiempo dedicado se pueden resumir en los siguientes tres grupos:

- **Incidencias por complejidad de la implementación:**

Errores en la implementación de carga de puntos de guardado y la implementación en el entrenamiento, y dimensiones en los valores calculados causaron gran impacto en las tareas 1.2,1.3 y 2.1.

- **Incidencias en el entrenamiento:**

Errores de parámetros, carga de datos y de modelos y de despliegue se terminaron por realizar tres tandas diferentes de entrenamientos, lo que causó gran impacto en las tareas 2.1,2.2,2.3 y 2.4.

- **Incidencias por coste computacional:**

Entrenar todas las funciones de pérdida resultó bastante más costoso de lo previsto inicialmente y causó gran impacto en las tareas 2.3 y 2.4.

Para resolver las desviaciones detectadas se aplicaron varias medidas correctoras:

- Inferencia de la arquitectura del modelo a partir del *state_dict* del *checkpoint*.
- Restauración de un método *forward* coherente con el utilizado durante el entrenamiento.
- Unificación de la función *run_inference* para evaluación y despliegue.
- Creación de un conjunto de test fijo denominado *representative_test*.
- Incorporación de un *baseline* de mezcla para validar los modelos frente a una solución trivial.
- Reentrenamiento mediante la versión V3 del protocolo experimental.
- Separación entre pérdidas principales, experimentales y avanzadas.

Aunque estas medidas provocaron un retraso respecto a la planificación inicial, permitieron establecer un proceso de evaluación más sólido obteniendo así mejores resultados. De este modo, se evitó seleccionar modelos únicamente por generar archivos de salida y se priorizó la obtención de resultados comparables, trazables y evaluables frente a un criterio común.

Siendo parte del proceso de la validación experimental, las incidencias ocurridas permitieron detectar y corregir errores que afectaban a la validez de los resultados

CAPÍTULO 3

Herramientas utilizadas



3.1. Python 3.10.19

Python es un lenguaje de alto nivel de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código. Se trata de un lenguaje de programación multiparadigma, ya que soporta parcialmente la orientación a objetos, programación imperativa y, en menor medida, programación funcional. Es un lenguaje interpretado, dinámico, multiplataforma.. Sus ventajas principales incluyen su sintaxis sencilla y legible (muy similar al lenguaje natural), su enorme ecosistema de librerías para Inteligencia Artificial y ciencia de datos.

3.2. Librosa

Librería de *Python* más popular para análisis de audio. Construida sobre *NumPy*, *SciPy* y *Matplotlib*. Utilizada para dar soporte a las transformaciones en dominio tiempo-frecuencia, visualización de ondas y espectrogramas, y cargas de audios.

3.3. Matplotlib

Librería estándar de visualización en *Python*. Proporciona principalmente herramientas para creación de gráficos. En el proyecto se ha utilizado para graficar señales, espectrogramas, valores de métricas y resultados.

3.4. Nussl (*Northwestern University Source Separation Library*)

Librería de Python especializada en separación de fuentes de audio, proporciona métodos de preprocesamiento de audio, implementación y comparación de algoritmos de separación de fuentes, modelos pre-entrenados, métricas ... En el proyecto se ha utilizado para cargas de audio *wav* y *mp3*, obtención y reconstrucción de espectrogramas , construcción del modelo, inferencia de las fuentes, generación de STFTs y obtención de métricas.

3.5. Pytorch

Librería de aprendizaje profundo en *Python* desarrollada por Meta (previamente *Facebook*) enfocada en la creación, entrenamiento y despliegue de redes neuronales. Se utiliza en el trabajo para definir el modelo de *MaskInference*, trabajar con los tensores, con las funciones de pérdida, para proporcionar métodos de entrenamiento, *backpropagation*, gestión del uso de CPU y carga y guardado de *checkpoints*.

Pythorch es prácticamente el núcleo del proyecto en cuanto a herramientas.

3.6. Ignite

Librería de alto nivel construida sobre *PyTorch* que facilita el entrenamiento de modelos mediante abstracciones que proporcionan métodos ya implementados para entrenamiento, validación , *checkpoints* ... Además de para generación de métricas, en el proyecto se utiliza principalmente para manejar bucles de entrenamiento/validación, guardado/carga de *checkpoints*, *early-stopping*, registro de métricas y *logs*.

3.7. Streamlit

Librería de *Python* pensada para creación de interfaces gráficas y aplicaciones *web* interactivas de manera simple y enfocada a ciencia de datos e IA. En el proyecto proporciona la implementación de lo que sería la parte del *Frontend* del proyecto.

3.8. Torchaudio

Librería oficial de *Python* centrada en el procesamiento de audio y voz proporcionando *datasets*, transformaciones, y modelos pre-entrenados. Se utiliza en

el proyecto para carga de audios, transformaciones, *Data Augmentation*, acceso al dataset *MUSDB...*

CAPÍTULO

Marco teórico

4



En esta sección se van a introducir una serie de conceptos teóricos considerados relevantes para el desarrollo del proyecto y que han requerido investigación y aprendizaje sobre ellos por ser un tema del que inicialmente se tenía poco conocimiento teórico.

Se divide el capítulo en dos grandes bloques, el primero dedicado a conceptos importantes en el campo del sonido y de las señales acústicas, y el segundo en relación al aprendizaje automático.

4.1. Sonidos y señales

Naturaleza del sonido

El sonido es una perturbación mecánica que se propaga en medios elásticos (aire, agua o sólidos) y que percibimos a través del sistema auditivo.

Algunos conceptos importantes:

■ 1. Ondas Sonoras

Las ondas sonoras son oscilaciones longitudinales que provocan variaciones en la presión del medio (ejemplo en Fig.1), zonas más densas (compresiones) y zonas menos densas (rarefacciones). Se generan por una fuente vibratoria, como una cuerda, un altavoz, cuerdas bucales o una gota de agua y esa vibración se transmite por colisión entre partículas del medio. [1]

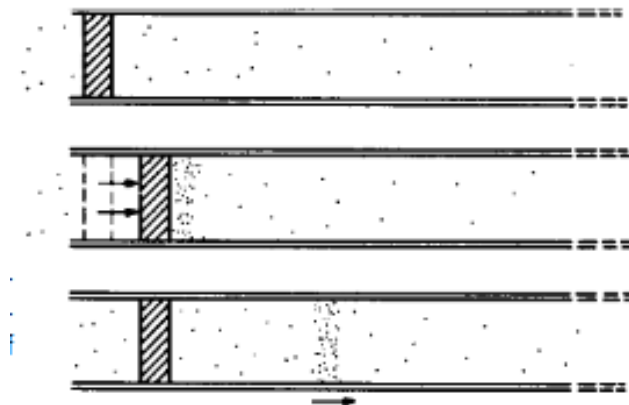


Figura 4.1: Generación de una onda sonora longitudinal mediante el movimiento de un pistón al final de un tubo [1].

■ 2. Frecuencia y tono

La frecuencia es el número de ciclos completos de la onda por segundo, medida en hercios (Hz) y determina el tono de lo que escuchamos. Las notas tienen sus propias frecuencias, por ejemplo $440Hz$ corresponde a *La4*, la voz humana suele ir en el rango 128-256 Hz , los ultrasonidos se consideran los superiores a 20 kHz y los infrasonidos los inferiores a 20 Hz [2]. El rango de audición humana oscila entre los 20 Hz y los 20 kHz .

Por otro lado, el tono está relacionado con la *frecuencia fundamental* (aunque en sonidos complejos intervienen también armónicos, que son múltiplos de la frecuencia fundamental, lo cual afecta también al timbre, siendo el timbre lo que nos permite distinguir una misma nota tocada en una guitarra, un violín, o una flauta.)

■ 3. Amplitud e intensidad

La amplitud corresponde al máximo desplazamiento de las partículas en el medio respecto a su posición de equilibrio, reflejándose en el sonido como la magnitud de variación de presión o vibración.

Se percibe como *volumen o sonoridad*, ondas con mayor amplitud suenan más fuertes.

La intensidad sonora mide la potencia transmitida por unidad de área (W/m^2) en la dirección del sonido y está relacionada con la amplitud al cuadrado[3]:

$$I \propto A^2$$

Dominio tiempo frecuencia

En esta sección se explicarán conceptos importantes para llevar a cabo el proyecto y relacionados con el dominio tiempo-frecuencia, como magnitud y fase, señales mono y estéreo, transformadas de Fourier y espectrogramas.

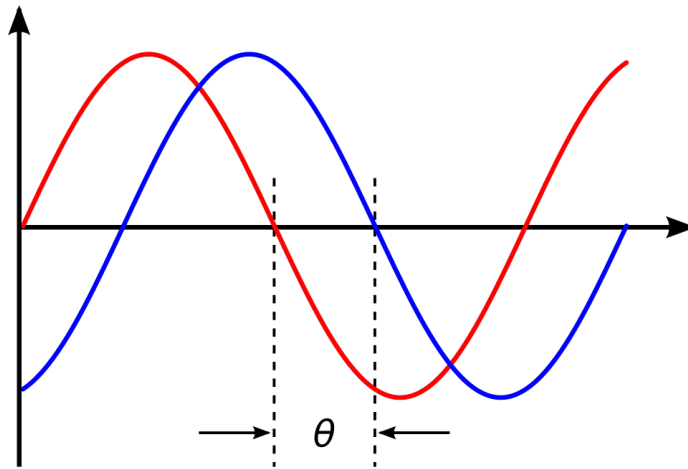
Magnitud y fase

La magnitud y fase son los dos principales componentes sonoros en los que descomponemos una señal. Se obtienen mediante la aplicación de la *transformada en tiempo corto de Fourier*. La magnitud es el módulo del coeficiente complejo de la STFT¹:

$$|X(t, f)|$$

Mientras que la fase determina el desfase temporal relativo de esa componente:

$$\angle X(t, f)$$



En la

Aunque la magnitud es frecuentemente usada como entrada a modelos de separación, la fase es esencial para reconstruir una señal con facilidad, y también la componente que más complicaciones supone, pues no siempre puede asumirse uniforme y se suele reutilizar la fase de la mezcla como aproximación[4].

¹Siglas para la transformada en tiempo corto de Fourier en inglés. Se explica en una sección más abajo que requiere de la explicación de la magnitud y fase.

Señales mono y estéreo

Son los dos tipos de señales sonoras (ejemplo en Figura 4.2). Las señales mono se presentan en un solo canal, mientras que las estéreo presentan dos canales, izquierdo y derecho, por lo que crean una sensación espacial y de amplitud dinámica.

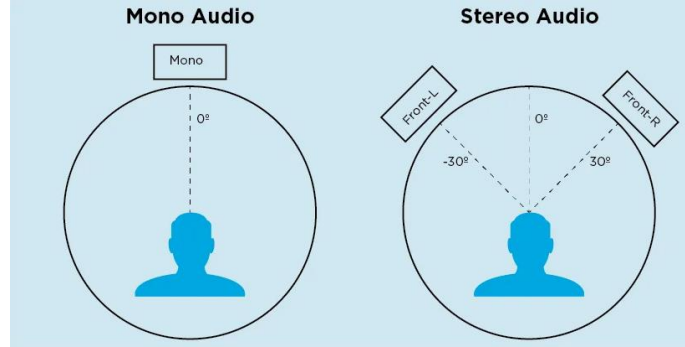


Figura 4.2: Representación gráfica de las diferencias entre sonido mono y sonido estéreo.

STFTs (Short-Time Fourier Transform / Transformada en Tiempo Corto de Fourier)

Se calcula sobre la representación en forma de onda del sonido computando una transformación discreta de Fourier (una transformación discreta de Fourier consiste en la captura mediante ventanas sucesivas, no necesariamente triangulares, pero sí normalmente, en movimiento L de secciones de la onda , Fig. 4.3) trasladada a lo largo de la onda (L , $2L$, $3L$...) Método que captura mediante ventanas sucesivas de diferentes formas secciones de una onda y que permite analizar la evolución de su frecuencia a lo largo del tiempo. Se define de acuerdo a la siguiente fórmula:

$$X(n, \omega) = \sum_{m=-\infty}^{\infty} x[m] \cdot w[m - n] \cdot e^{-j\omega m}$$

Donde:

- $x[m]$ es la señal de audio original en tiempo discreto. Cada valor representa una muestra digital de la señal.
- m es el índice de muestra utilizado para recorrer la señal.

- $w[m-n]$ es la función de ventana desplazada hasta la posición temporal n . Esta ventana selecciona el fragmento de la señal que se analiza en cada instante.
- n representa la posición temporal de la ventana dentro de la señal.
- $e^{-j\omega m}$ es una exponencial compleja utilizada para analizar la presencia de una determinada frecuencia en el fragmento seleccionado.
- j es la unidad imaginaria, definida mediante $j^2 = -1$.
- ω es la frecuencia angular digital, normalmente expresada en radianes por muestra.

Cada celda de la transformada es compleja (contiene magnitud y fase) y la transformada en general es invertible, por lo que se puede obtener la onda inicial a partir de la representación.

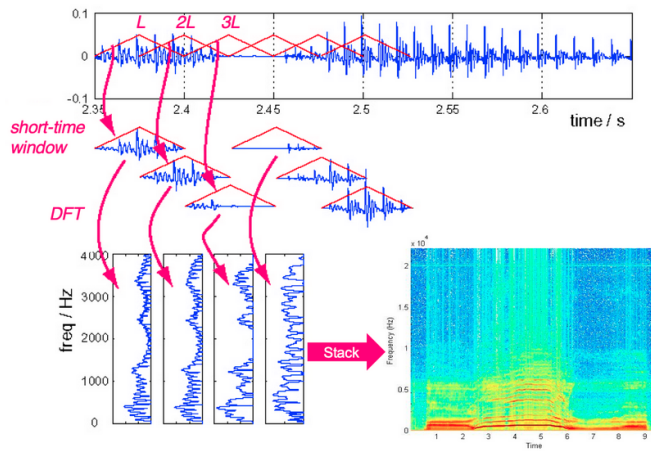


Figura 4.3: Proceso de computar una transformada en tiempo corto de Fourier a partir de una onda. Imagen de Bryan Pardo

[5]

Un parámetro importante en la transformada de Fourier es la forma de la de ventana que se utiliza (gaussiana, rectangular, triangular, cuadrática ...), la longitud de la ventana y la distancia entre ventanas adyacentes. En el proyecto llevado a cabo se utiliza *sqrthann*. En la Figura 4.4 se puede ver el desempeño en separación de fuentes según las diferentes ventanas utilizadas.

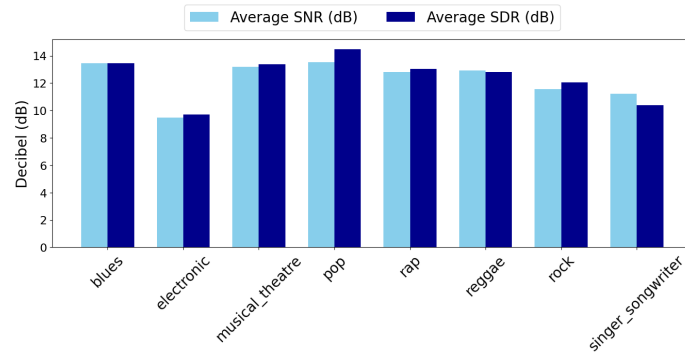


Figura 4.4: Hann function

Más parámetros que pueden variar en función de lo que se busque en una transformada de fourier:

- 1. *window length*: tamaño de la ventana.
- 2. *hop length*: salto entre ventanas consecutivas
- 3. Solapamiento: solapamiento entre ventanas.

Existe también el término *iSTFT* que refiere al proceso inverso de la transformada de fourier, donde se reconstruye una señal de audio u onda a partir de su representación en el dominio tiempo-frecuencia

Espectrogramas

Es la representación visual, como ejemplifica la Figura 4.5 de la magnitud de la STFT, y contrapone gráficamente tiempo y frecuencia haciendo un gradiente de color a más oscuro según se incrementa la más intensidad/magnitud presente la señal en ese punto.

Existen diferentes tipos, en función de cómo se traten los respectivos elementos de la STFT (magnitud , cuadrado, logarítmico, logarítmico cuadrado)

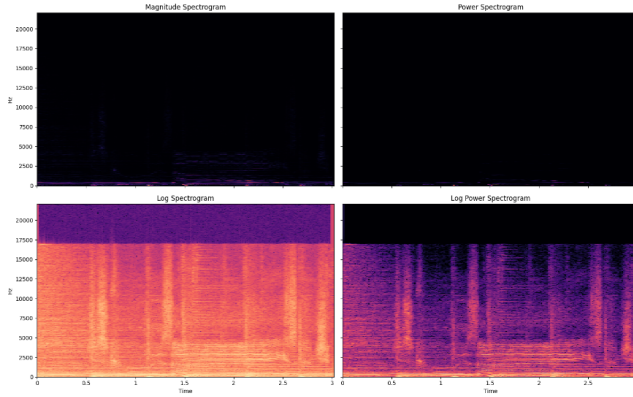


Figura 4.5: Diferentes generaciones de espectrogramas según el tratamiento de la STFT.

[5]

Mezcla de señales y separación de fuentes

En esta subsección se comentarán las diferentes metodologías de mezcla aprendidas para entender el funcionamiento de la mezcla de señales sonoras.

Mezclas lineales

Una mezcla lineal de señales se produce cuando varias fuentes $s_i(t)$ se suman con diferentes pesos a_i o coeficientes:

$$x(t) = \sum_{i=1}^N a_i \cdot s_i(t)$$

En música esto sucede al grabar instrumentos o voces con micrófonos con distinto valor de ganancia² En música esto ocurre cuando diferentes instrumentos o voces se graban a través de micrófonos cada uno con diferente ganancia, esto simplifica la realidad física de como el sonido se propaga y se capta y ha facilitado el desarrollo de muchos de los modelos actuales con este objetivo.

En casos reales es mas complicado. La música real incluye paneo (por dónde se orienta el sonido, en auriculares, por ejemplo, izquierdo o derecho), reverberación (fenómeno acústico en el que un sonido persiste ligeramente después de que la fuente original ha dejado de emitirlo), efectos, retardos (como el eco)...

²La ganancia en un micrófono es un ajuste que controla el nivel de la señal de entrada en el preamplificador antes de procesarse o grabarse. El preamplificador es un dispositivo electrónico que eleva el voltaje bajo de la señal.

Mezclas coherentes e incoherentes

Las mezclas coherentes son aquellas que estan correlacionadas/son dependientes en tiempo y frecuencia,fase y estructura. Por ejemplo , dos guitarras tocando el mismo *riff* , o un coro cantando al unísono.

Las mezclas incoherentes son aquellas que tienen baja correlación, por ejemplo, una progresión de acordes en tonalidad grave y una melodía tocada por una guitarra en tonalidad aguda, que en el espectro de frecuencias ocuparán rangos distintos.

Estas últimas son mas fáciles de separar por lo general ya que se puede diseñar un algoritmo que aproveche que ocupan espacios distintos el espectro frecuencial.

Representaciones digitales

En esta sección se explicarán las diferentes representaciones digitales del sonido y cómo se han utilizado y tenido en cuenta en el proyecto.

Formatos del audio

Existen varias formas de guardar una representación digital del sonido:

- **Formatos sin compresión:** *WAV*, *AIFF*. Formatos que guardan directamente las muestras de audio codificadas, ofrecen mayor fidelidad al original pero también generan archivos más pesados. Es el formato utilizado en todo el proyecto.
- **Otros formatos:** como formatos comprimidos sin pérdida (*FLAC*,*ALAC*... reducen el tamaño sin alterar información) o formatos comprimidos con pérdida (*MP3*,*AAC*,*OGG*... descartan información que el oído humano no percibe)
- **Espectrogramas:** ya definidos en una sección anterior.

En el proyecto, el sistema trabaja con espectrogramas de magnitud lineal obtenidos mediante STFT ya que posteriormente se reconstruye el resultado obtenido mediante iSTFT.

Aunque por lo general en procesamiento de audio se utilizan otro tipo de espectrogramas (espectrogramas de Mel, explicados en el siguiente párrafo) ya que el objetivo del modelo es estimar máscaras tiempo-frecuencia que posteriormente se aplican sobre la magnitud STFT de la mezcla para reconstruir las fuentes usando la fase de la mezcla.

Un espectrograma de Mel es similar a un espectrograma en el sentido de que muestra el contenido de frecuencia de una señal de audio a lo largo del tiempo, pero en un eje de frecuencia diferente.En un espectrograma estándar, el eje de frecuencia es lineal y se mide en hercios (Hz). Sin

embargo, el sistema auditivo humano es más sensible a los cambios en las frecuencias bajas que en las frecuencias altas, y esta sensibilidad disminuye de manera logarítmica a medida que la frecuencia aumenta. La escala mel es una escala perceptual que aproxima la respuesta de frecuencia no lineal del oído humano. [6]. En la Figura 4.6 se muestra la diferencia en la representación visual de ambos métodos.

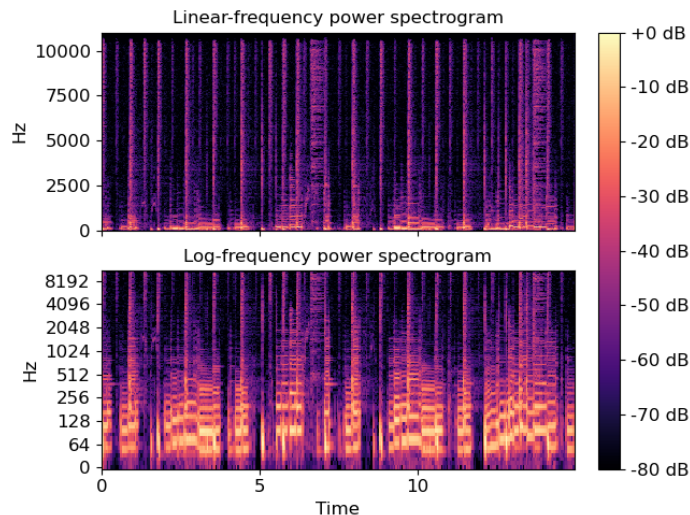


Figura 4.6: Espectrograma estándar arriba y espectrograma de Mel abajo.

Tasa de muestreo

La tasa de muestreo determina la fidelidad y el rango de frecuencias que puede representarse:

- **44.1 kHz:** estándar para *CDs*. Permite cubrir hasta 22.05 kHz, que está por encima del rango auditivo humano.
- **48 kHz:** usado en producción audiovisual, audio para video y transmisión digital.
- **96 kHz y 192 kHz:** considerados *alta resolución*, aunque la diferencia es mínima para el oyente, se utilizan en investigación y procesamiento para evitar *aliasing* o realizar transformaciones más detalladas.

4.2. Aprendizaje automático

Conceptos fundamentales

En esta sección se comentarán conceptos relativos al marco teórico del segundo ámbito de la ingeniería tocado en este proyecto, el aprendizaje automá-

tico. Se comentarán conceptos fundamentales generales, conceptos fundamentales sobre las redes neuronales artificiales y sobre las técnicas de evaluación y métricas utilizadas en el proyecto. Finalmente se comentará el estado del arte actual en lo relativo al objetivo del trabajo.

Funciones de pérdida

Las funciones de pérdida son parte crucial del aprendizaje automático. El término refiere a una función matemática que estima el error entre la estimación y el objetivo. En función de la función de pérdida elegida, se prioriza un tipo de error u otro. Por ejemplo:

- Función de pérdida L1, error absoluto medio: minimiza el error absoluto promedio entre predicción y objetivo.

$$\mathcal{L}_{L1}(\hat{Y}, Y) = \frac{1}{N} \sum_{i=1}^N |\hat{Y}_i - Y_i|$$

Donde Y_i representa el valor real de la magnitud de la fuente en una celda tiempo-frecuencia, $\hat{Y}_i$ representa el valor estimado por el modelo y N es el número total de elementos comparados.

- Función de pérdida L2, error cuadrático medio: minimiza el error cuadrático medio.

$$\mathcal{L}_{L2}(\hat{Y}, Y) = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i - Y_i)^2$$

Donde Y_i representa el valor real de la magnitud de la fuente, $\hat{Y}_i$ representa el valor estimado por el modelo y N es el número total de elementos comparados.

Aprendizaje supervisado vs no supervisado

El aprendizaje supervisado y no supervisado son dos enfoques fundamentales en el campo del aprendizaje automático, cada uno con sus propias características, aplicaciones y desafíos, ejemplo en Fig.5 . El aprendizaje supervisado se basa en la utilización de datos etiquetados para entrenar modelos que puedan hacer predicciones o clasificaciones precisas. En contraste, el aprendizaje no supervisado se enfoca en descubrir patrones y estructuras ocultas en datos

no etiquetados, lo que lo hace especialmente útil para tareas como la agrupación y la reducción de dimensionalidad.

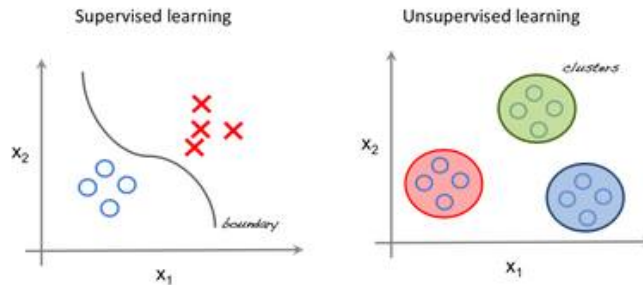


Figura 4.7: Comparativa de clasificación entre aprendizaje supervisado y no supervisado. Imagen de Juan Domingo Farnos

[5]

Overfitting, underfitting y generalización

Otros dos conceptos importantes a la hora de trabajar con algoritmos de inferencia de clases, son el overfitting y el underfitting. El overfitting ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento, capturando ruido o patrones específicos que no generalizan bien a nuevos datos. Esto puede resultar en un rendimiento excelente en el conjunto de entrenamiento pero pobre en datos no vistos. Por otro lado, el underfitting sucede para el caso contrario, cuando un modelo es demasiado simple para capturar la complejidad de los datos, lo que lleva a un rendimiento deficiente tanto en el conjunto de entrenamiento como en datos nuevos. Finalmente otro concepto importante es la generalización. Esta se refiere a la capacidad de un modelo para desempeñarse bien en datos no vistos, y es crucial encontrar un equilibrio entre overfitting y underfitting para lograr una buena generalización.

Dataset

El concepto de dataset refiere al conjunto de datos intencionalmente elegidos para ser utilizados en el entrenamiento, validación o prueba de un modelo de aprendizaje automático. Un dataset bien diseñado es crucial para el éxito de cualquier proyecto de aprendizaje automático, ya que la calidad y representatividad de los datos pueden influir significativamente en el rendimiento del modelo. A modo de ejemplo, en el contexto de este proyecto se han utilizado dos datasets diferentes para los procesos de entrenamiento-validación y evaluación, junto con un conjunto de test fijo adicional. La intención detrás de esta elección es garantizar que el modelo se entrene y valide con datos que reflejen una amplia variedad de casos, mientras que la evaluación se realiza con un conjunto de datos completamente independiente para medir la capacidad de

generalización del modelo. Esta estrategia ayuda a evitar problemas como el overfitting, donde el modelo se ajusta demasiado a los datos de entrenamiento y no generaliza bien a nuevos datos.

RNNs artificiales

Tipos de redes

Sin entrar mucho en detalle, aquí se mencionan las arquitecturas de redes neuronales más comunes, cada una con sus propias características y aplicaciones específicas.

- **Redes Neuronales Feedforward (FNNs):** Son las redes neuronales más simples, donde la información fluye en una sola dirección, desde las capas de entrada hasta las capas de salida, sin ciclos ni conexiones recurrentes. Son adecuadas para tareas de clasificación y regresión.
- **Redes Neuronales Convolucionales (CNNs):** Están diseñadas para procesar datos con una estructura de cuadrícula, como imágenes. Utilizan capas convolucionales para extraer características espaciales y son ampliamente utilizadas en visión por computadora.
- **Redes Neuronales Recurrentes (RNNs):** Son adecuadas para procesar secuencias de datos, como texto o series temporales. Tienen conexiones recurrentes que permiten mantener información a lo largo del tiempo, lo que las hace ideales para tareas como el procesamiento del lenguaje natural y la predicción de secuencias.
- **Redes de Memoria a Corto y Largo plazo (LSTMs):** Variante de RNN donde se introduce una celda estado que retiene información importante a lo largo de toda la secuencia. Diseñadas para procesar datos secuenciales (como texto o audio).

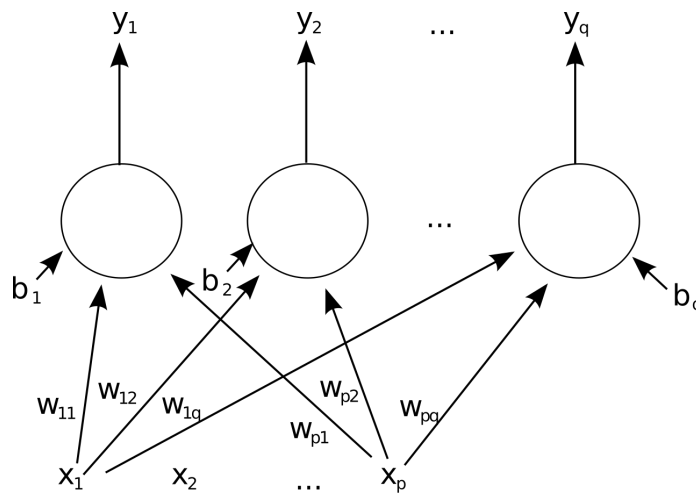


Figura 4.8: Una sola capa de red neuronal artificial feed-forward. Por Mcstrother - Trabajo propio, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=9515940>
[5]

Optimización y descenso de gradiente

Para mejorar una red neuronal, nuestro objetivo es encontrar pesos y sesgos que minimicen la función de coste. Como las redes neuronales suelen tener un gran número de pesos y sesgos, suele tratarse de un problema de optimización de gran dimensión. Para minimizar la función de coste, comentamos a raíz de que es el método utilizado en el proyecto el descenso de gradiente. El descenso de gradiente es un algoritmo de optimización iterativo que se utiliza para encontrar los valores de los parámetros (pesos y sesgos) que minimizan una función de coste. El algoritmo funciona calculando el gradiente de la función de coste con respecto a los parámetros y luego actualizando los parámetros en la dirección opuesta al gradiente, lo que conduce a una disminución de la función de coste. Este proceso se repite hasta que se alcanza un mínimo local o global de la función de coste, lo que indica que se han encontrado los mejores parámetros para el modelo.

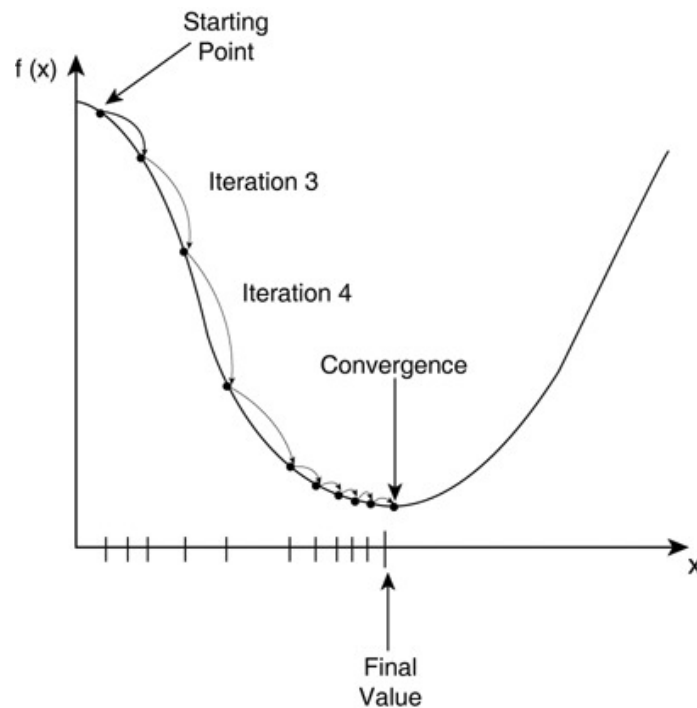


Figura 4.9: Ilustración del proceso de descenso de gradiente.
[5]

Neuronas, capas y funciones de activación y pérdida

Otros conceptos a tener en cuenta:

- Neuronas: unidades básicas de procesamiento que reciben entradas, aplican una función de activación y generan una salida.
- Capas: agrupaciones de neuronas que pueden realizar diferentes transformaciones en sus entradas.
- Funciones de activación: funciones matemáticas que determinan si una neurona debe activarse o no, introduciendo no linealidad en la red.

Funciones de pérdida

En el proyecto se comparan diferentes funciones de pérdida con el objetivo de analizar como afecta la medición del error al entrenamiento del modelo de separación.

Para cada una, se compara la estimación generada por la red con una referencia y cada una se centra en aspectos diferentes de la representación espectral o de la reconstrucción.

Siendo:

- Y la magnitud objetivo de la fuente.

- $\hat{Y}$ la magnitud estimada.

- X la magnitud de la mezcla.

- $\hat{M}$ la máscara estimada.

- N el número de elementos comparados.

- ϵ constante para evitar división o logaritmos de cero.

Las funciones son:

4. MARCO TEÓRICO

Función de pérdida	Comentario	Expresión matemática
L1	Penaliza el error absoluto entre la magnitud estimada y la magnitud objetivo. Es una pérdida simple y robusta frente a errores extremos.	$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N \hat{Y}_i - Y_i $
L2	Penaliza el error cuadrático entre la estimación y la referencia. Da más peso a errores grandes que la pérdida L1.	$\mathcal{L}_{L2} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i - Y_i)^2$
L1 frecuencia	Variante de L1 aplicada sobre una reorganización frecuencial de los tensores, tratando cada fuente del batch como una muestra independiente.	$\mathcal{L}_{L1freq} = \frac{1}{N} \sum_{i=1}^N \hat{Y}_i^{freq} - Y_i^{freq} $
L2 frecuencia	Variante de L2 aplicada sobre la representación frecuencial. Mantiene la penalización cuadrática tras reformular las dimensiones de los tensores.	$\mathcal{L}_{L2freq} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i^{freq} - Y_i^{freq})^2$
Log L1	Calcula el error absoluto y lo expresa en una escala logarítmica. Reduce el efecto del rango dinámico de las magnitudes.	$\mathcal{L}_{logL1} = \frac{1}{B} \sum_{b=1}^B \log_{10} \left(\sum_i \hat{Y}_{b,i} - Y_{b,i} + \epsilon \right)$
Log L2	Calcula el error cuadrático y lo transforma a escala logarítmica. Penaliza errores grandes, pero con una escala comprimida.	$\mathcal{L}_{logL2} = \frac{1}{B} \sum_{b=1}^B \log_{10} \left(\sum_i (\hat{Y}_{b,i} - Y_{b,i})^2 + \epsilon \right)$
Log magnitud	Compara la magnitud estimada y la magnitud objetivo después de aplicar una transformación logarítmica. Permite trabajar con diferencias relativas.	$\mathcal{L}_{logMag} = \frac{1}{N} \sum_{i=1}^N \left \log_{10}(\hat{Y}_i + \epsilon) - \log_{10}(Y_i + \epsilon) \right $
Log-compressed L2	Calcula una pérdida cuadrática entre magnitudes log-comprimidas. Su objetivo es reducir el rango dinámico antes de comparar estimación y referencia.	$\mathcal{L}_{logCompL2} = \frac{1}{N} \sum_{i=1}^N \left(\log(\hat{Y}_i + \epsilon) - \log(Y_i + \epsilon) \right)^2$
LPSA	En la implementación utilizada, compara espectros de potencia en escala logarítmica. Permite trabajar con diferencias relativas de energía espectral.	$\mathcal{L}_{LPSA} = \frac{1}{N} \sum_{i=1}^N \left(\log(\hat{Y}_i^2 + \epsilon) - \log(Y_i^2 + \epsilon) \right)^2$
Mask L1	Compara la máscara estimada con una máscara ideal obtenida a partir de las magnitudes objetivo de las fuentes.	$M_i^{ideal} = \frac{Y_i}{\sum_s Y_{i,s} + \epsilon}, \quad \mathcal{L}_{mask} = \frac{1}{N} \sum_{i=1}^N \hat{M}_i - M_i^{ideal} $

Cuadro 4.1: Pérdidas principales consideradas durante el entrenamiento.

Pérdidas avanzadas o experimentales

Función de pérdida	Comentario	Expresión matemática
LPSA phase	Variante que incorpora información de fase de la mezcla y de la fuente objetivo. Requiere disponer de las fases θ_X y θ_Y .	$Y_i^{PSA} = Y_i \cos(\theta_{X,i} - \theta_{Y,i}), \quad \mathcal{L}_{LPSAphase} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i - Y_i^{PSA})^2$
LMRS	Pérdida basada en la relación logarítmica entre fuente y mezcla. Compara ratios espectrales en lugar de magnitudes absolutas.	$\mathcal{L}_{LMRS} = \frac{1}{N} \sum_{i=1}^N \left(\log \frac{Y_i + \epsilon}{X_i + \epsilon} - \log \frac{\hat{Y}_i + \epsilon}{\hat{X}_i + \epsilon} \right)^2$
L-MRS	Pérdida multirresolución calculada tras reconstruir señales temporales. Combina términos espectrales en distintas resoluciones STFT.	$\mathcal{L}_{MRS} = \frac{1}{R} \sum_{r=1}^R (SC_r(\hat{y}, y) + \mathcal{L}_{logMag,r}(\hat{y}, y))$
Deep-feature	Compara activaciones internas extraídas por una red auxiliar a partir de los espectrogramas estimado y objetivo.	$\mathcal{L}_{DF} = \sum_{j \in J} \ \phi_j(\hat{Y}) - \phi_j(Y)\ _1$
Deep-feature-EMD	Variante experimental que compara distribuciones de activaciones profundas mediante una aproximación regularizada de la distancia Earth Mover.	$\mathcal{L}_{DF-EMD} = \sum_{j \in J} \text{Sinkhorn}(\mu_{\phi_j(\hat{Y})}, \mu_{\phi_j(Y)})$

Cuadro 4.2: Pérdidas avanzadas y experimentales consideradas durante el desarrollo.

CAPÍTULO

Estado del arte 5



Dentro del ámbito de la inteligencia artificial , la separación de fuentes musicales es un campo que ha experimentado una evolución desde sus primeros acercamientos en 1973 [5] utilizando un *vocoder homomórfico* para separar la función de excitación y la respuesta al impulso del tracto vocal. Estos métodos se clasifican en tres ramas principales: procesamiento de señales , modulación de audio o discurso y teoría de la probabilidad (acercamientos guiados por datos).

En el procesamiento de señales, se busca un filtrado de la mezcla, ya sea aplicando un filtro de *pasa baja*, *pasa alta* o *de banda* a la mezcla completa , o mediante la aplicación de máscaras en rango [0-1] de ganancias a aplicar a cada elemento en una representación tiempo-frecuencia del audio.

En la modulación de audio/discursos, se estudia la predictibilidad del comportamiento de las señales haciendo diferenciación entre *señales sinusoidales* (o tonos puros) y *ruido* , siendo ambos los extremos en una línea de predictibilidad , por un lado el comportamiento de las señales sinusoidales puede ser predicho en un futuro a partir de muestreos previos , mientras que por otro , el *ruido blanco* se define como una señal impredecible a futuro y su espectrograma presenta energía constante en todo el rango frecuencial. Dentro de esta categoría se presentan varias herramientas , como *cepstrum* , con aplicaciones de estimación de frecuencias fundamentales , derivación de frecuencias cepstrales , y filtrado de señal.

En los acercamientos probabilísticos, se diferencia el *Modelo oculto de Markov* que sin entrar en mucho detalle , es útil para observaciones variantes en el tiempo y el *modelo de mezcla Gaussiano* que da una forma de aproximar una distribución como una suma ponderada de *gaussianos* , útil para crear máscaras.

5.1. Separación de fuentes musicales

La separación de fuentes musicales consiste en descomponer una mezcla musical en varias señales individuales, por ejemplo, voz, batería, bajo y otros.

Una mezcla musical puede entenderse como combinación de varias fuentes: voz, bajo, batería, instrumentos efectos... Se suelen contemplar dos escenarios:

- Separación en dos fuentes: voz y acompañamiento. - Separación en cuatro fuentes: voz, bajo, batería y otros.

Las lista aplicaciones [5] de esta descomposición en fuentes de una pista musical es bastante amplia, desde karaokes, producción musical, *remixing* de piezas antiguas, educación musical.. hasta análisis automático de música, y restauración o edición de audio.

De todas formas, por las características de distribución frecuencial de los elementos en el espectro de frecuencias, el tipo de pieza musical... existen una serie de grupos principales que engloban los principales problemas que se suelen enfrentar al intentar dar una solución metódica al problema:

- Solapamiento espectral: los elementos presentes en una obra musical suelen presentar solapamiento en el espectro frecuencial, los elementos (voces, instrumentos, baterías) no ocupan un rango único en el espectro y esto dificulta su identificación.
- Información de fase: la información de fase es crucial para la calidad de la separación, pero es difícil de modelar y recuperar.
- Mezclas comerciales con efectos: los efectos que se utilizan en las mezclas comerciales dificultan esta tarea; efectos como la reverberación, retardo, compresión... modifican como se comportan los elementos de una obra frente a aquellas que no presentan este tipo de efectos, dificultado la tarea de identificación de estos.
- Simultaneidad de instrumentos: lo más común en la música es tener dos, tres, cuatro... instrumentos que armónicamente se complementan y suenan a la vez. Esto también dificulta la identificación y posterior separación de cada uno de ellos.

5.2. Representación tiempo-frecuencia y máscaras

Una representación tiempo-frecuencia codifica el espectro variable en el tiempo [7] y es utilizada por muchos métodos de separación ya que las fuentes tienden a estar menos solapadas en esta representación. La STFT (Short-Term Fourier Transform, Transformada de Fourier de ventana corta en español) es la representación más común, ya que permite invertirse para volver a la forma de onda. La magnitud de la STFT se denomina espectrograma. Un espectrograma es una representación visual de una señal acústica y representa el espectro de

frecuencias de una señal a lo largo del tiempo. Esta idea es especialmente relevante ya que suele ser la representación que se utiliza en casi todos los métodos actuales.

Por otro lado las máscaras tiempo-frecuencia (en cuya generación se centra el modelo que se utiliza en el proyecto) son matrices de valores en el rango $[0, 1]$ que se generan con el objetivo de aplicarse a cada celda tiempo-frecuencia de la mezcla original para obtener una aproximación de la fuente y reconstruirla posteriormente mediante una transformación inversa [5]. El trabajo presentado no predice la señal de cada fuente si no que genera una máscara tiempo-frecuencia que aplicada sobre la magnitud STFT de la mezcla genera una aproximación de la fuente.

5.3. Métodos clásicos de separación

Los métodos clásicos de separación de fuentes musicales se pueden agrupar en tres categorías principales: modelado de la señal principal, modelado del acompañamiento, y acercamientos conjuntos y probabilísticos

Modelado de la señal principal

En este enfoque se busca modelar la voz o melodía principal como una señal armónica, para, posteriormente, substraer esa señal de la mezcla original y obtener el acompañamiento [5]. La voz cantada se produce por vibración de cuerdas vocales, y suele presentar frecuencia fundamental y armónicos, por lo que los métodos más clásicos intentaban estimar el *pitch* y reconstruir o filtrar la señal principal. En la cita utilizada para informarse de este tema, se resume esta familia de métodos como métodos basados en la hipótesis de harmonicidad: primero se estima la frecuencia fundamental de la señal principal y después se obtiene separación por resíntesis o filtrado. La estructura más común suele ser:

- Hipótesis: la señal principal suele ser armónica.
- Se estima la frecuencia fundamental.
- Se localizan armónicos.
- Se reconstruye o filtra la fuente principal.

Limitaciones comunes: el método falla si la voz no es puramente armónica, si hay fonemas sordos, saturaciones, no se identifica bien la voz o no se detecta bien la frecuencia fundamental. También dificultan el método los casos en los que los instrumentos son armónicos o la voz pueda estar muy mezclada con reverberación o efectos.

Modelado del acompañamiento

Este enfoque es un poco la idea opuesta al anterior: se modela el acompañamiento para posteriormente substraerlo de la mezcla original y obtener la

voz.

El acompañamiento suele ser más redundante o repetitivo que la señal principal: patrones rítmicos, armónicos, o estructurales que suelen repetirse en el tiempo. Esta redundancia permite modelarlo y separarlo, muchos acercamientos clásicos basados en el modelado del acompañamiento toman ventaja de esta característica: la redundancia, se aprovechan componentes de bajo rango, voz dispersa, y repeticiones internas del acompañamiento.

Entre las limitaciones de este acercamiento se encuentran que: no todo acompañamiento es repetitivo, la existencia de canciones con cambios estructurales fuertes, existencia de elementos solistas o improvisados que producen imprevisibilidad o presencia de repetibilidad en el elemento vocal.

Acercamientos conjuntos y probabilísticos

En este enfoque no se modelan solo voz o acompañamiento y posteriormente se extrae el contrario de la mezcla original. En este caso se busca modelar ambos de forma conjunta.

Entre los modelos que históricamente han presentado mejor desempeño se encuentran:

- Modelo oculto de Markov (HMM): modelo estadístico que analiza secuencias de datos donde los estados reales no se pueden observar directamente, pero que se pueden inferir a través de efectos observables y medibles. Dada esta definición, el modelo oculto de Markov es útil para el problema de la separación de fuentes para observar variantes en el tiempo.
- Modelo de mezcla Gaussiana (GMM): modelo probabilístico para agrupar datos o estimar densidades. En el caso de la separación de fuentes se utiliza como forma de aproximar distribuciones mediante sumas ponderadas Gaussianas.

Ambos modelos se utilizan para asignar valores a celdas tiempo-frecuencia de fuentes. Modelan estados o distribuciones que pueden ayudar a estimar fuentes o máscaras y no siempre asignan directamente celdas TF.

Entre las limitaciones de estos acercamientos encontramos: requerimiento de hipótesis explícitas previas, posible dificultad de ajustar parámetros, requerimiento de un modelo estadístico que aproxime bien la música real...

5.4. Enfoques basados en aprendizaje profundo

El aprendizaje profundo es uno de los campos que más se ha desarrollado en la actualidad gracias a los avances en la capacidad computacional y la existencia de bases de datos cada vez más grandes. Si bien existen sistemas de aprendizaje profundo cuya entrada y salida consisten directamente en onda acústica, estos métodos no son los más populares debido al requerimiento de arquitecturas complejas y su alto coste computacional. Si bien existen bases de datos grandess (la mas larga actualmente de diez horas) , estas no son suficientes para que un modelo de estas cracterístias sea capaz de producir

satisfactoriamente una pista de audio de salida donde únicamente este presente un solo elemento de la composición inicial.

La tecnología más prolífica, por los motivos mencionados en el párrafo anterior son las redes neuronales, se basa en encadenar una serie de transformaciones aprendidas en un entrenamiento para ser aplicadas en la señal de entrada, aprendiendo así representaciones y *mapeos* que permitan obtener resultados mediante cantidades grandes de datos de aprendizaje.[5]

Modelos basados en espectrograma

Muchos modelos neuronales toman como entrada una magnitud STFT o un espectrograma, y producen como salida una máscara o una magnitud estimada.

- Entrada: magnitud de mezcla.
- Modelo: red neuronal.
- Salida: máscara por cada fuente.
- Reconstrucción: máscara sobre magnitud de mezcla mas fase de mezcla.

Una referencia muy buena es la implementación de *Open-Unmix*: implementación abierta para separación musical basada en redes neuronales [8].

Modelos *waveforms* e híbridos

La evolución moderna en este campo no se limita solamente al uso de espectrogramas, extiende también modelos que trabajan directamente con ondas o que combinan ambas representaciones, ondas y espectrogramas. Por ejemplo, *Hibryd Demucs* [9] combina separación basada en espectrograma y en onda, y se presenta como separación híbrida en ambos dominios donde el modelo decide cuál de ellos puede ser más útil dependiendo del caso o incluso combina ambos [9].

5.5. Herramientas y tecnologías actuales

Actualmente gracias al gran desarrollo de las capacidades de computo y el desarrollo de bases de datos amplias, existen tecnologías que dan muy buenos resultados, en el contexto de los métodos anteriormente mencionados, los métodos que aparecen y reinantes de la actualidad corresponderían a la categoría de modelos guiados por datos.

Algunos de los más interesantes:

- **Open-Unmix:** implementación de red neuronal referente en el campo de la separación musical. Provee de modelos preparados para su uso que permite separación de pistas pop en cuatro fuentes: vocales, baterías, bajo y resto. Se trata de un modelo abierto basado en deep learning y que permite reproducibilidad [8].
- **Spleeter:** conjunto de modelos pre-entrenados de separación de fuentes en pista musicales. Basados en redes neuronales convolucionales con salto de conexiones (que se traducen en codificadores-decodificadores llamados *U-Nets*) de doce capas, utilizando función de pérdida *L1-norm*, entrenados con bases de datos internas de *Deezer* y con espectrogramas de la mezcla como entrada [10].
- **Demus / Hybrid Demucs:** modelo avanzado que, en función del tipo de pista de entrada, toma como entrada su espectrograma, su onda, o ambos. De los estados más modernos en el campo [9].

Además, mencionar también **MUSDB18** que es un dataset/benchmark de referencia que contiene 150 conjuntos de mezcla y cuatro fuentes comprendiendo bajo, batería, voz y resto. Permite comparar sistemas y está dividido en entrenamiento y test. Ha sido el *dataset* utilizado en este trabajo de fin de grado.

5.6. Posicionamiento del trabajo

Habiendo ya enumerado los métodos clásicos y más actuales del campo, y teniendo en cuenta su alto desempeño en esta tarea, es difícil con los conocimientos de principiante en el campo dar unos resultados mejores. Este TFG no busca superar modelos del estado del arte como *Spleeter*, *Open-Unmix* o *Demucs* si no que se limita a dar un acercamiento inicial "sencillo" para aprender sobre la construcción de los sistemas que dan solución a este problema y dar un estudio experimental controlado dentro del alcance al estado del arte sobre diferentes funciones de pérdida.

En concreto se ha enfocado en lo siguiente: implementar un proceso completo, controlado y reproducible. Trabajar con STFTs y máscaras. Comparar el desempeño de diferentes funciones de pérdida. Evaluar dos y cuatro fuentes, implementar una interfaz de uso fácil y entendible para el usuario y analizar problemas reales de entrenamiento, inferencia y evaluación. En resumen, no se busca superar el estado del arte, si no simplemente proporcionar una comparativa de diferentes funciones de pérdida y construir un proceso completo.

Parte 1 : separación de fuentes

CAPÍTULO 6

Para el objetivo de la identificación y separación de elementos en una obra musical, se ha utilizado una arquitectura *LSTM*. Particularmente la en *Open-Unmix* [5] y de cuyo artículo de presentación donde existe una sección didáctica de donde se ha aprendido gran parte del método usado.

Existen dos escenarios de separación:

- 2 fuentes: voz/acompañamiento
- 4 fuentes: voz/bajo/batería/otros

En dos fuentes normalmente el acompañamiento suele ser suma de bajo+batería+otros.

6.1. Diseño del modelo

Tras un estudio en cierta profundidad de los métodos comunmente utilizados para el objetivo que nos concierne ¡Se llega a la conclusión individual de que existen gran cantidad de métodos diversos! [7]

En nuestro caso, y aprovechando el recurso en formato tutorial que presentan los autores anteriormente citados en esta sección, y tras una prueba inicial de generación de máscaras, se elige el modelo utilizado previamente por los autores cuyo funcionamiento apunta a la inferencia de máscaras tiempo-frecuencia para la separación de las fuentes.

A tener en cuenta:

- El modelo no separa directamente onda temporal, si no que predice máscaras tiempo-frecuencia.

- La salida estimada es una magnitud que se calcula como máscara de salida por magnitud de la mezcla.
- En el entrenamiento se comparan magnitudes estimadas con magnitudes objetivo.
- En el proceso de inferencia de la máscara se reconstruye el audio usando la fase de la mezcla.

6.2. Arquitectura base

El modelo empleado es una arquitectura recurrente de inferencia de máscaras, basada en una LSTM que recibe la magnitud de la STFT de la mezcla y predice una máscara por fuente. Estas máscaras se aplican sobre la magnitud de la mezcla para obtener las magnitudes estimadas de cada *stem*.

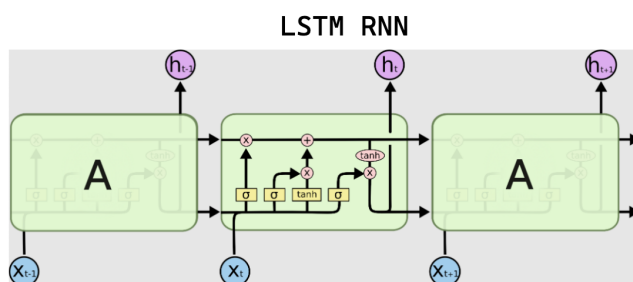


Figura 6.1: Ejemplo de una arquitectura RNN-LSTM
[5]

En la figura se muestra una red neuronal recurrente LSTM desplegada en el tiempo.

Una LSTM procesa una secuencia paso a paso, palabras por ejemplo, sonidos o datos temporales.

Cada bloque verde representa la misma celda LSTM aplicada en diferentes momentos en el tiempo x_t y x_{t+1}

x_t : es la entrada en el instante actual.

h_t : es el estado oculto o salida de la celda en ese instante, lo que la red "entiende" en ese momento

Línea horizontal superior : es el estado oculto o salida de la celda en ese instante. Resume lo que la red ha *entendido* hasta el momento.

Bloque central, contenidos :

- Puerta de olvido: decide qué información antigua se elimina de la memoria.

- Puerta de entrada: decide qué información nueva se guarda.
- Candidato de memoria: genera nueva información posible para añadir a la memoria.
- Puerta de salida: decide qué parte de la memoria se usa para producir h_t

Los símbolos **sigmoid**, **tanh** son funciones que controlan la cantidad que pasa de una información concreta. Por ejemplo, **sigmoide**, produce valores continuos en (0,1) y hace la función de una "perilla." o "compuerta suave" para la información.

Además, se aplica el concepto de bidireccionalidad donde la información "viaja en dos direcciones". Esto quiere decir que se tiene en cuenta el contexto pasado y futuro de la entrada.

En el contexto del objetivo de utilizar esta arquitectura en el proyecto: el sistema introduce al modelo la interpretación en secuencia temporal de un espectrograma (es decir, la magnitud de la STFT de la entrada), a partir del cuál, la LSTM procesa esta secuencia para estimar máscaras.

6.3. Modificaciones propuestas

Arquitectura y parámetros

En cuanto a proponer un nuevo enfoque a un problema mediante inteligencia artificial, generalmente buena parte de los métodos tratan de aportar alguna configuración de parámetros o modificación a la arquitectura que resulten en unos mejores resultados.

- Arquitectura:

La arquitectura base no fue sustituida por un modelo diferente, sino que se estabilizó su implementación. Durante la fase de depuración se detectó que la ruta de inferencia debía coincidir exactamente con la utilizada durante el entrenamiento. Por ello, se mantuvo la estructura basada en inferencia de máscaras, restaurando el preprocesado mediante conversión a decibelios y normalización por lotes antes de la red recurrente. Esta corrección permitió que la LSTM recibiera una representación coherente con la usada durante el entrenamiento, siendo la siguiente:

STFT → magnitud → AmplitudeToDB → BatchNorm → LSTM bidireccional → Embedding → máscaras → iSTFT

- Parámetros:

Para los parámetros, se tuvo que realizar una segunda versión de estos, ya que, tras considerar una tanda de entrenamientos realizada como válida, al evaluar se llegó a la conclusión de que los parámetros del modelo

base y los parámetros de nuestros entrenamientos no coincidían, lo que llevó a incompatibilidad entre *checkpoints*, configuración y *forward* y por ello, se decidió realizar una segunda tanda de entrenamientos con los parámetros corregidos que se muestran a continuación:

Parámetro	Valor
Dominio de trabajo	Tiempo-frecuencia, mediante STFT
Tamaño de ventana STFT	512 muestras
Salto STFT	128 muestras
Tipo de ventana	sqrt_hann
Número de bins de frecuencia	257
Canales de entrada	1, mono
Tipo de red recurrente	LSTM
Número de capas recurrentes	1
Tamaño oculto	50
Bidireccional	Sí
Dropout	0.0
Activación de salida	Sigmoid
Salidas en 2 stems	vocals, accompaniment
Salidas en 4 stems	vocals, bass, drums, other

Cuadro 6.1: Configuración arquitectónica base empleada en los entrenamientos V2.

Función de pérdida: *Deep-feature-EMD*

La función de pérdida *Deep-feature-EMD*, propuesta por el tutor de este proyecto donde se hace uso de una distancia tipo *Sinkhorn/EMD* sobre las características extraídas. Define la parte mas experimental del proyecto.

La pérdida *DeepFeature-EMD* compara la estimación y el objetivo no directamente en el dominio de magnitud, sino en el espacio de activaciones de una red convolucional auxiliar. Para cada capa considerada, las activaciones se aplanan y se interpretan como distribuciones[11]. La discrepancia entre ambas distribuciones se mide mediante una distancia Sinkhorn, de la Earth Mover's Distance, que es aquí donde recibe la titularidad de experimental y también la de propuesta del tutor La pérdida transforma estimación y objetivo en representaciones espectrales, extrae activaciones mediante una CNN auxiliar y compara las distribuciones de activaciones usando una distancia Sinkhorn, aproximación regularizada de EMD.

El extractor (CNN auxiliar) no está entrenado en música, por lo que la pérdida se considera experimental y no necesariamente correlaciona con calidad perceptual.

6.4. Configuración de entrenamiento

Inputs y targets

La entrada principal al modelo es la magnitud de la mezcla en dominio STFT, en particular:

STFT compleja + magnitud + tensor

Internamente se adapta al formato $[B, T, F]$, se convierte a dB y se normaliza antes de entrar a la RNN. La magnitud lineal se conserva para aplicar la máscara al final.

Los targets (objetivos) son las magnitudes de las fuentes reales. Existen dos modalidades de salida:

- **4 fuentes:** *vocales + batería + bajo + resto*
- **2 fuentes:** *vocales , acompañamiento*

La estructura de entrada al modelo es:

$$[B, T, F]$$

Donde:

- B : tamaño del batch, es decir, número de ejemplos procesados simultáneamente.
- T : número de frames temporales del espectrograma.
- F : número de bins de frecuencia obtenidos mediante la STFT.

Y la estructura de salida/*targets*:

$$[B, T, F, C, S]$$

Donde:

- B : tamaño del batch.
- T : número de frames temporales.
- F : número de bins de frecuencia.
- C : número de canales de audio. En este trabajo se utiliza $C = 1$, ya que las señales se procesan en mono.

- S : número de fuentes o *stems* a separar. En este trabajo puede ser $S = 2$ para voz y acompañamiento, o $S = 4$ para voz, bajo, batería y resto.

La salida del modelo no genera audio directamente, aunque en la ejecución lo parezca la salida son máscaras tiempo-frecuencia. Posteriormente, se aplica la máscara sobre la magnitud de la mezcla y el resultado corresponde a una estimación de la magnitud de la mezcla. Luego para reconstruir el audio se utiliza la fase de la pista original. Esto es una limitación importante, ya que es una estimación. La fase de la mezcla original no tiene porque ser la de las fuentes, pero a falta del dato de la magnitud para estas últimas, se utiliza la de la mezcla.

TFMs aplicadas

En esta sección se comentarán las transformaciones (TFMs) aplicadas al audio/tensores antes del entrenamiento:

De forma general, el flujo de procesamiento puede resumirse como:

$$\text{audio} \rightarrow \text{STFT} \rightarrow \text{magnitud y fase}$$

$$\text{magnitud} \rightarrow \text{AmplitudeToDB} \rightarrow \text{BatchNorm} \rightarrow [B, T, F] \rightarrow \text{LSTM}$$

$$\text{máscara estimada} \times \text{magnitud de mezcla} + \text{fase de mezcla} \rightarrow \text{iSTFT}$$

Donde cada transformación es:

- **Transformaciones espectrales.**

En primer lugar, la señal de audio se transforma desde el dominio temporal al dominio tiempo-frecuencia mediante la STFT (*Short-Time Fourier Transform*). Esta transformación divide la señal en ventanas temporales sucesivas y aplica una transformada de Fourier a cada una de ellas, obteniendo una representación compleja que indica el contenido frecuencial de la señal en cada instante temporal.

A partir de la STFT se separan dos componentes principales:

- **Magnitud:** corresponde al módulo de cada coeficiente complejo de la STFT. Representa la intensidad relativa de cada componente frecuencial en cada frame temporal. En este trabajo, la magnitud de la mezcla se utiliza como entrada principal del modelo, mientras que las magnitudes de las fuentes reales se utilizan como objetivos durante el entrenamiento.

- **Fase:** corresponde al ángulo de cada coeficiente complejo de la STFT. Contiene información sobre el desplazamiento temporal relativo de las componentes frecuenciales. El modelo no estima directamente la fase de cada fuente, por lo que durante la reconstrucción se reutiliza la fase de la mezcla como aproximación.

En el caso de separación en dos fuentes, las fuentes del dataset pueden agruparse para formar los objetivos *vocals* y *accompaniment*. En este caso, el acompañamiento se obtiene a partir de la combinación de las fuentes no vocales, como bajo, batería y resto.

■ **Transformaciones internas del modelo.**

Una vez obtenida la magnitud de la mezcla, se aplican varias transformaciones internas antes de introducirla en la red recurrente:

- **AmplitudeToDB:** convierte la magnitud lineal a una escala logarítmica expresada en decibelios. Esta transformación comprime el rango dinámico de la señal, reduciendo la diferencia entre componentes de muy alta y muy baja energía. Esto facilita que el modelo trabaje con valores más estables durante el entrenamiento.
- **BatchNorm:** la normalización por lotes normaliza las activaciones utilizando estadísticas calculadas sobre el batch. En este trabajo se aplica después de la conversión a decibelios y antes de la LSTM, con el objetivo de estabilizar el entrenamiento y facilitar la convergencia del modelo.
- **Reordenación dimensional:** las operaciones de carga de datos, cálculo de STFT y procesamiento con redes recurrentes pueden requerir distintas ordenaciones de los ejes del tensor. Por ello, antes de introducir los datos en la LSTM, la magnitud se adapta al formato $[B, T, F]$, donde B es el tamaño del batch, T el número de frames temporales y F el número de bins de frecuencia. De esta forma, el espectrograma se interpreta como una secuencia temporal de vectores frecuenciales.

La LSTM procesa esta secuencia temporal y produce una representación a partir de la cual se estiman máscaras tiempo-frecuencia para cada fuente. Estas máscaras se aplican sobre la magnitud de la mezcla para obtener las magnitudes estimadas de los distintos *stems*.

■ **Transformaciones para pérdidas concretas.**

Algunas funciones de pérdida requieren adaptar la forma de los tensores antes de calcular el error entre la estimación y el objetivo. Estas transformaciones no modifican el contenido de la señal, sino la organización

de sus dimensiones para adecuarlas al tipo de comparación que realiza cada pérdida.

- **Pérdidas de frecuencia:** en las pérdidas que comparan directamente representaciones frecuenciales, las estimaciones y los objetivos pueden reformularse al formato $[B \cdot S, F, T]$, donde S es el número de fuentes. De este modo, cada fuente de cada ejemplo del batch se trata como una muestra independiente en el cálculo de la pérdida.
- **Pérdidas *deep-feature*:** en las pérdidas basadas en características profundas, las magnitudes estimadas y objetivo se adaptan al formato $[B \cdot S, 1, F, T]$. Esta forma permite introducir los espectrogramas en una red convolucional auxiliar, donde el eje 1 representa un único canal de entrada y las dimensiones F y T se tratan como una representación bidimensional tiempo-frecuencia.

Preprocesado de un *dataset*

En esta sección se explicará el camino desde los archivos de audio hasta los *batches* de entrenamiento.

- **1. Organización inicial del dataset:** El dataset se compone de mezclas y fuentes separadas. Para cada pista se tiene:
 - *mixture.wav*: la mezcla original.
 - *vocals.wav*: pista para las vocales.
 - *drums.wav*: pista para la batería.
 - *bass.wav*: pista para el bajo.
 - *other.wav*: pista para elementos extra.
- **2. Construcción de escenarios:** Como ya se ha comentado en secciones anteriores, el modelo contempla dos escenarios de salida: cuatro y dos pistas. Partiendo de ello, el dataset de entrada para los dos tipos de entrenamiento debe ser acorde a la salida. Para ello:
 - *2 stems*: se mantienen las vocales y se agrupa el resto como acompañamiento.
 - *4 stems*: se usan las cuatro fuentes originales.
- **3. Conversión a dominio espectral:** Se calcula la STFT de mezcla y fuentes con los mismos parámetros. Se extraen magnitudes para entrenamiento y se conserva la fase de la mezcla para reconstrucción/evaluación.
- **4. Preparación de *batches*:** Cada *batch* contiene al menos:

- *mix_magnitude*
- *source_magnitudes*

Y para pérdidas que piden fase:

- *mixture_phase*
- *source_phase*

La versión final del proceso de entrenamiento comprueba la presencia de estos dos últimos para las funciones que las piden, y usa `source_magnitudes` como *target* principal durante el entrenamiento.

■ **5. División train/validation/test:**

- *train/validation*: usado durante entrenamiento y *early stopping*.
- *representative_test*: conjunto fijo usado solo para evaluación final.

División 60-40¹.

- **6. Normalización/canales:** El sistema se configuró para un canal de audio, lo que reduce el coste computacional y se recomienda en la documentación del modelo[7]. También simplifica el proceso de comparación entre fuentes.

6.5. Entrenamiento del modelo

Esta sección explicará bajo qué lógica se entrenó, el qué se entrenó, qué observaciones salieron durante el proceso y responderá a las preguntas:

- ¿Qué experimentos se entrenaron?
- ¿Por qué se organizaron así?
- ¿Cómo ha sido la evolución experimental?

Resultados preliminares y experimentales

Los entrenamientos fueron agrupados por funciones de pérdida, en particular por una estructura de tipos dentro de estas para ayudar a su posterior documentación y evaluación:

- **Pérdidas principales:** *L1*, *L2*, *L1_freq*, *log L1*, *log L2*, *log_mag*, *log_compressed_l2*, *LPSA*, *mask_L1*.
- **Pérdidas experimentales:** *deep_feature*, *deep_feature_emd*.
- **Pérdidas avanzadas:** *l_mrs*, *lpsa_phase*.

¹60 % train 40 % validation

También existe la división que ya se explicó anteriormente entre dos y cuatro fuentes. El caso de las dos fuentes representa una separación más sencilla y cercana a escenarios de voz y acompañamiento. El caso de cuatro fuentes introduce mayor complejidad, ya que el modelo debe repartir la mezcla entre varias fuentes instrumentales con solapamiento espectral. Se respetó la estructura que presenta el *dataset* utilizado de *MUSDB18*, compuesto por 150 pistas con *stems* aislados, con etiquetas en el orden de *vocales, baterías, bajos y otros* y dividido en conjuntos de entrenamiento y test.

Como conclusiones preliminares podemos comentar:

- Las primeras versiones del entrenamiento permitieron detectar problemas de formato, inferencia y evaluación.
- Las pérdidas simples o directas sobre magnitud, son más fáciles de interpretar y sirven como base comparativa.
- Las pérdidas experimentales basadas en características profundas, tienen mayor coste y mayor incertidumbre, porque dependen de su extractor auxiliar.
- Las pérdidas avanzadas con fase o reconstrucción temporal son más complejas y, por tanto, conviene añadir control sobre ellas, piden cautela.

Comparativas

Aquí vamos a hacer una comparativa preliminar del conjunto de funciones presentado junto con sus versiones de dos y cuatro fuentes.

■ A. Comparación por número de fuentes

La documentación utilizada y los recursos citados en este proyecto en múltiples ocasiones comentan las diferencias entre estos dos enfoques al problema. La separación que resulta en el conjunto de dos fuentes, ya visto en las primeras versiones del entrenamiento, es un enfoque más sencillo que permite una "binarización" del problema al reducirse a dos salidas. En contraposición con el segundo conjunto, donde la separación se extiende a identificar cuatro elementos en el espectro, los elementos que se puedan identificar si el método es válido pueden variar enormemente dificultando así la identificación de estos, ya que, un elemento que a lo mejor está en un rango frecuencial concreto en una mezcla, en otra puede habitar en otro no a lo mejor completamente diferente pero sí lo suficiente para que la sensibilidad de la función a estos cambios no dé al modelo la información necesaria para ser capaz de identificar un elemento que varía. Por esto, es esperable que, por lo general, 4 *stems* obtenga peores métricas en la fase de evaluación. Identificar y separar dos elementos en contraposición y por lo comentado, resulta en una tarea más

sencilla ya que la variabilidad se reduce a eso, dos elementos, mientras que el caso anterior eleva la premisa de la necesidad de sensibilidad en la función al doble de elementos donde puede variar este rango frecuencial.

Por ejemplo, géneros como la electrónica o el rap que se basan en elementos frecuencialmente bajos (bajo, sub-bajo) presentan gran superposición entre fuentes como bajo y baterías, es un sonido más "sucio" armónicamente complejo. Por otro lado, por ejemplo, el blues, pop, o canciones de autor, presentan mejores desempeños en la separación por sus características de armónicos vocales e instrumentales más limpios [12].

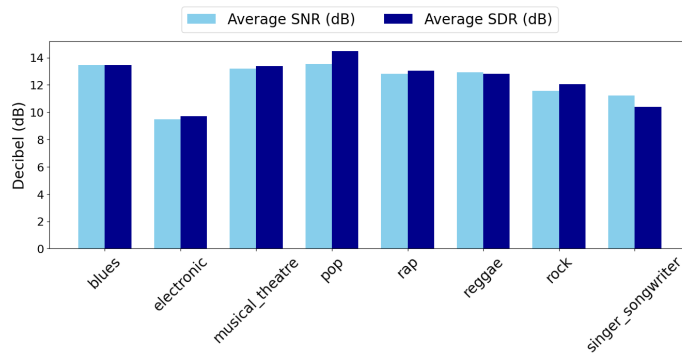


Figura 6.2: Average SNR and SDR by genre

■ B. Comparación por familia de pérdida

Se han agrupado las funciones de pérdida por las familias aritméticas a las que pertenecen por su naturaleza:

- Pérdidas de magnitud directa: L1, L2.
- Pérdidas frecuenciales: L1_freq, L2_freq.
- Pérdidas logarítmicas: logL1, logL2, log_mag, log_compressed_l2.
- Pérdidas informadas por fase o potencia: LPSA, lpsa_phase.
- Pérdidas basadas en máscara: mask_l1
- Pérdidas experimentales: deep_feature, deep_feature_emd

Las distintas familias de pérdidas introducen sesgos de entrenamiento diferentes. Las pérdidas en escala logarítmica trabajan sobre una representación comprimida de la magnitud espectral, ya que la aplicación del logaritmo reduce el rango dinámico del espectrograma [13]. Esto modifica el peso relativo de los errores, evitando que los bins de mayor energía dominen completamente la optimización. Las pérdidas frecuenciales comparan directamente estimaciones y referencias en el dominio

tiempo-frecuencia, normalmente mediante distancias punto a punto entre espectrogramas [14]. Por su parte, las pérdidas basadas en máscara emplean como objetivo una máscara ideal o aproximada, guiando al modelo hacia una asignación explícita, aunque generalmente suave, de los bins tiempo-frecuencia a las distintas fuentes [15, 16].

Reentrenamiento V2

Esta versión nace como corrección de la primera versión .

La primera versión, como ya se comentaba en el punto anterior, entrenaba con señales estéreo sin haber cerrado todavía el criterio de canales. Esto hacía que los tensores de entrada y salida no estuvieran alineados con la arquitectura final planteada..

V2 reentrena los modelos con una configuración mas coherente, se entrenan modelos de dos y cuatro fuentes de salida, y sirve para obtener una primera batería de serie de *checkpoints*. Durante la primera evaluación de V2 , se detectan nuevos problemas: carga de checkpoints, diferencias entre entrenamiento e inferencia, stems muy correlacionados con la mezcla y resultados que no superaban *baseline*. Finalmente, y por estos motivos, se termina por considerar el conjunto de modelos entrenados como insuficiente, aunque no se descarta completamente y se deja en el repositorio a modo de marca temporal para mostrar la evolución del proyecto.

Reentrenamiento V3 y protocolo final

El reentrenamiento numero tres corresponde a la versión actual y a la que ha proporcionado los resultados que se muestran en esta memoria para comparar el desempeño de las funciones de pérdida. Nace en respuesta a lo visto y aprendido en la versión anterior.

En esta versión se introduce un conjunto de test fijo, se añade una *baseline* trivial de mezcla y se unifica la inferencia entre evaluación y despliegue , como cambios principales.

Y más en profundidad:

- Se restaura el *forward* coherente con el entrenamiento que se había modificado en la versión anterior a raíz de sospechar que ese era el punto que no permitía una generación de resultados correcta:

AmplitudeToDB + BatchNorm + RNN + Embedding

- Se separan pérdidas principales y experimentales.
- Se organiza la salida en un nuevo directorio.
- Se guarda un sumario de configuración de entrenamiento por modelo. Esto ayuda a hacer un seguimiento a posteriori.

- Se ajusta la planificación por coste final

Parte 2 : interfaz

CAPÍTULO 7



La interfaz visual actúa como herramienta demostrativa y didáctica. Permite aplicar modelos entrenados, visualizar resultados y facilitar el uso del sistema.

7.1. Objetivos

Interacción y aprendizaje

Los objetivos principales de la implementación para dar capacidades de interacción y aprendizaje se pueden resumir así:

- Cargar audio.
- Seleccionar modelo.
- Elegir función de pérdida/modelo entrenado. Proporcionar información de la elección.
- Seleccionar número de fuentes. Proporcionar información sobre la decisión.
- Separar fuentes.
- Reproducir resultados.
- Descargar stems.
- Visualizar waveform y espectrograma
- Opcionalmente entrenar un modelo propio con posibilidad de variar hiperparámetros. Proporcionar información sobre estos.

7.2. Diseño de experiencia de usuario

Organización y visualización

Los elementos presentes en la interfaz visual son los siguientes.

Barra lateral de configuración. La barra lateral permite seleccionar el número de fuentes, el modelo disponible, la base de datos, la función de pérdida y el checkpoint que se desea cargar.



Figura 7.1: Barra lateral de configuración de la interfaz.

Zona principal de carga. La zona principal permite subir un archivo de audio y lanzar el proceso de separación.



Figura 7.2: Zona de carga de la GUI.

Consola y logs. La consola muestra información sobre la ejecución del sistema, incluyendo mensajes de carga, separación, evaluación y posibles errores.

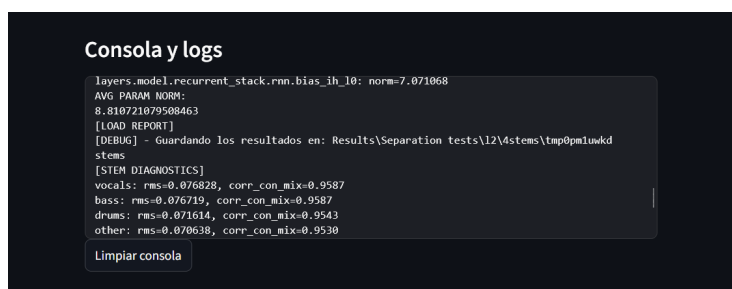
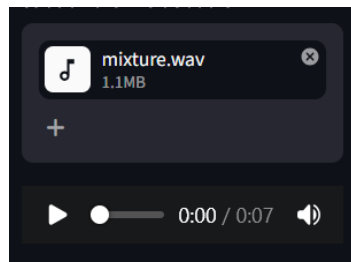
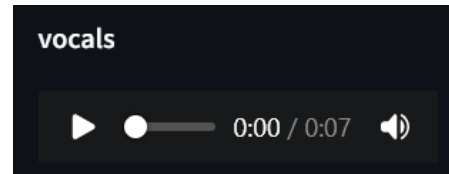


Figura 7.3: Consola y logs de la GUI.

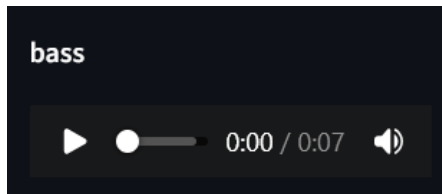
Reproductores. Tras la separación, la interfaz muestra reproductores de audio para la mezcla original y para cada una de las fuentes separadas.



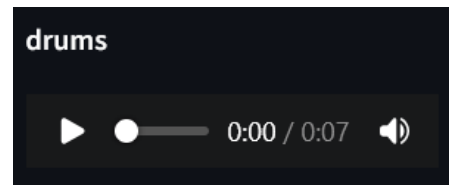
(a) Mezcla original.



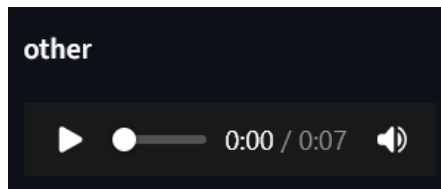
(b) Vocales.



(c) Bajo.



(d) Batería.



(e) Otros.

Figura 7.4: Reproductores de audio de la interfaz.

Gráficos. La interfaz también permite visualizar representaciones gráficas de las fuentes separadas, como la forma de onda y el espectrograma.

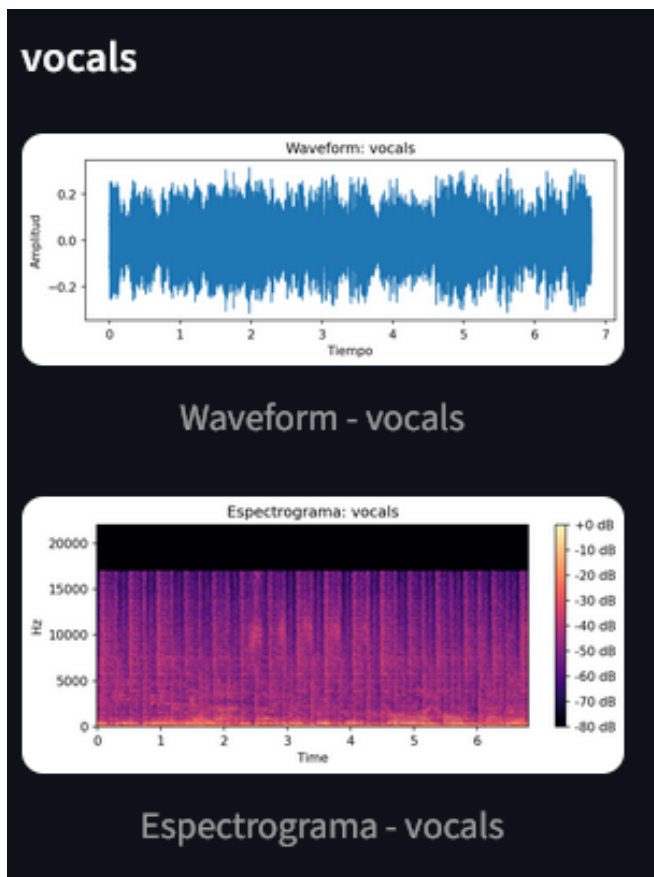
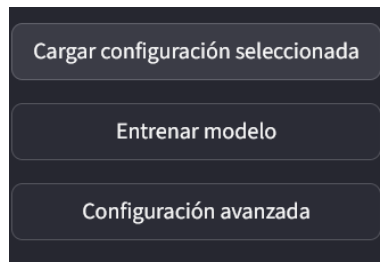
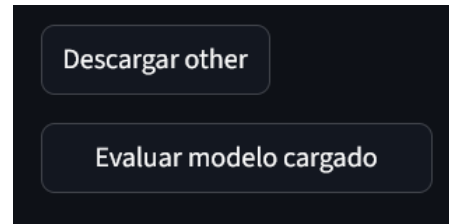


Figura 7.5: Ejemplo de representación gráfica de la fuente vocal.

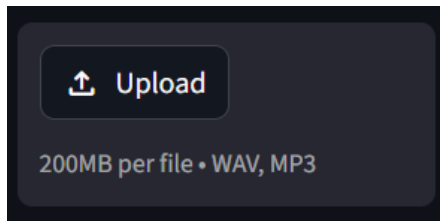
Botones. La interfaz incluye botones para lanzar la separación, evaluar el modelo y descargar los audios generados.



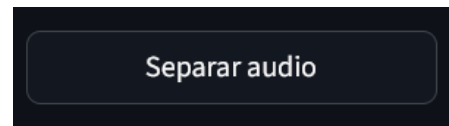
(a) Botón de separación.



(b) Botón de evaluación.



(c) Botón de descarga.



(d) Botón auxiliar.

Figura 7.6: Botones principales de la interfaz.

Explicaciones y guías

Al hacer click en el siguiente botón:

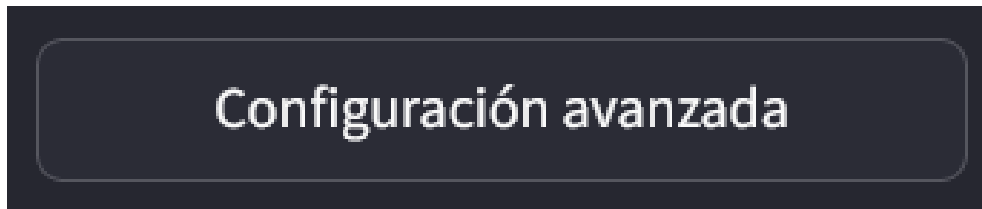


Figura 7.7: Botón de configuración.

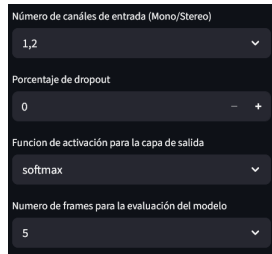
El sistema lleva al usuario a una nueva pantalla donde se pueden modificar los hiperparámetros del modelo:

Parámetros de la formación de transformadas de Fourier



Figura 7.8: Parámetros de formación de transformadas de Fourier modificables en la interfaz visual.

Parámetros intrínsecos modelo



Número de canales de entrada (Mono/Stereo)

1,2

Porcentaje de dropout

0

Funcion de activación para la capa de salida

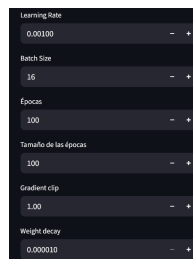
softmax

Numero de frames para la evaluación del modelo

5

Figura 7.9: Parámetros intrínsecos del modelo modificables en la interfaz visual.

Parámetros de entrenamiento



Learning Rate

0.00100

Batch Size

16

Épocas

100

Tamaño de las épocas

100

Gradient clip

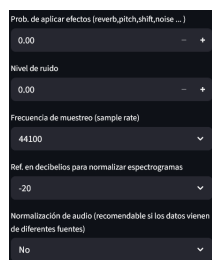
1.00

Weight decay

0.000010

Figura 7.10: Parámetros modificables de entrenamiento de nuevos modelos en la interfaz visual.

Parámetros de efectos para entrenamiento y de audio



Prob. de aplicar efectos (reverb, pitch, shift, noise ...)

0.00

Nivel de ruido

0.00

Frecuencia de muestreo (sample rate)

44100

Ref. en decibelios para normalizar espectrogramas

-20

Normalización de audio (recomendable si los datos vienen de diferentes fuentes)

No

Figura 7.11: Parámetros modificables de generación de efectos en la interfaz visual.

Parámetros de generación de mezclas



Figura 7.12: Parámetros modificables de generación de mezclas en la interfaz visual.

7.3. Casos de uso

Entrenamiento parametrizable

El sistema permite entrenar modelos al usuario utilizando el sistema definido en la sección **Parte 1**.

A partir de los parámetros definidos en configuración avanzada, se hace llamada al mismo método que se ha utilizado para los entrenamientos de los modelos del proyecto, y se comienza un nuevo ciclo de iteraciones de entrenamiento. Este nuevo modelo guardará sus *checkpoints* en una carpeta a parte de la de los modelos entrenados previamente para no pisar el trabajo realizado. Esto permite al usuario experimentar con el entrenamiento, ver cómo puede afectar la variación de hiperparámetros en los resultados y mostrar a alguien ajeno al tema qué es realmente un sistema de aprendizaje automático y , aunque no es un proceso instantáneo, y seguramente el usuario no llegue a esperar lo suficiente para obtener un modelo propio entrenado que ha aprendido lo suficiente como para generar resultados interesantes, se ha añadido esta funcionalidad porque se considera interesante que el usuario pueda ver en tiempo real cómo es realmente un entrenamiento de un modelo de inteligencia artificial.

Parámetros principales ajustables

El sistema permite el ajuste de una serie de parámetros para permitir al usuario jugar con estos y ver sus efectos en el modelo entrenado:

Parámetros de la formación de transformadas de Fourier

- Tamaño de la ventana STFT.
- Desplazamiento entre ventanas STFT.
- Tipo de ventana utilizada para la STFT.

Parámetros intrínsecos del modelo

- Número de canales de entrada (Mono / Estéreo).

- Porcentaje de *dropout*.
- Función de activación para la capa de salida.

Parámetros de efectos para entrenamiento y de audio

- Prob. de aplicar efectos (reverb,pitch,shift,noise ...).
- Nivel de ruido.
- Frecuencia de muestreo (sample rate).
- Ref. en decibelios para normalizar espectrogramas
- Normalización de audio (recomendable si los datos vienen de diferentes fuentes).

Parámetros de entrenamiento

- *Learning Rate*.
- *Batch Size*.
- Épocas.
- Tamaño de las épocas.
- *Gradient clip*.
- *Weight decay*

Parámetros de generación de mezclas

- *Coherent prob*

Además, se introduce otra pestaña adicional, donde se da información sobre lo que es cada parámetro



(a) Botón para acceder a la ventana de información sobre los parámetros

(b) Captura sobre la información de los parámetros

Figura 7.13: Entrada a la ventana de información sobre los parámetros de la configuración avanzada

Visualización de gráficos y espectrogramas

El sistema muestra por pantalla una serie de visualizaciones una vez se separa una mezcla con el objetivo de proporcionar inspección visual de los resultados:

Waveforms: una vez se separa una mezcla, se presenta por pantalla la forma de onda de cada una de las fuentes y de la mezcla. Ver figura.

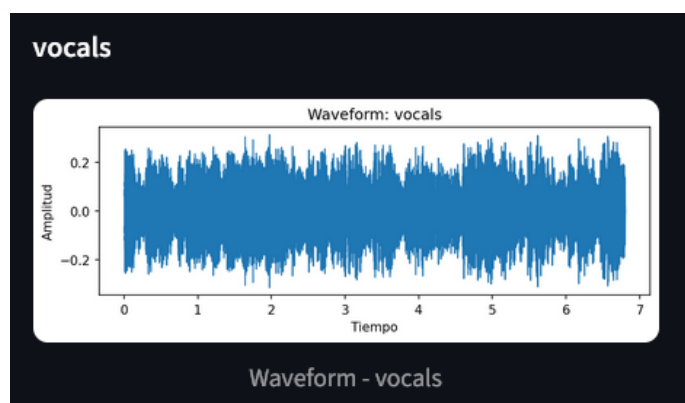


Figura 7.14: Ejemplo de una forma de onda mostrada en la interfaz al finalizar la separación de una pista.

Espectrograma: a la vez que se muestra la forma de onda de la mezcla y de cada fuente, se muestra en conjunto el respectivo espectrograma. Ver figura.

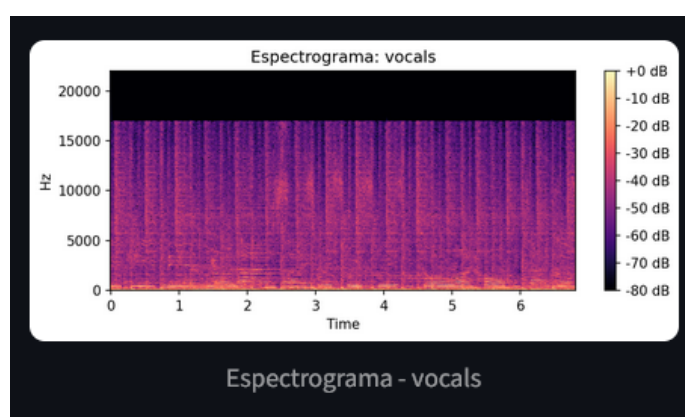


Figura 7.15: Ejemplo de un espectrograma mostrado en la interfaz al finalizar la separación de una pista.

Feedback

El sistema devuelve al usuario datos sobre las acciones que toma cuando estas generan salidas en la consola, se permite realizar una acción, se está realizando una acción o se necesita información que el usuario medio pueda desconocer.

Logs de consola: cuando el sistema realiza un proceso, lo más seguro es que este imprima por pantalla *logs* que puedan resultar informativos. Ver figura.

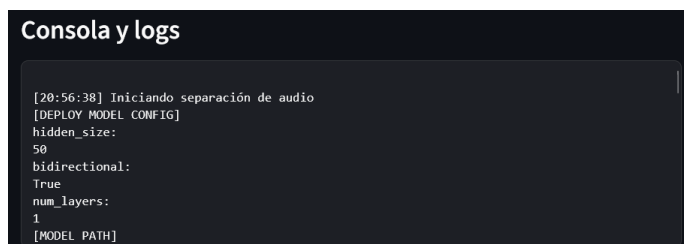


Figura 7.16: Ejemplo de *logs* mostrados en la consola de la interfaz

Mensajes de error: el usuario puede realizar alguna acción que requiera algún paso previo. Si bien no se ha hecho un estudio exhaustivo de todas las situaciones que puedan desencadenar un error, el sistema muestra mensajes de error en caso de que se produzca un error contemplado. Ver figura.

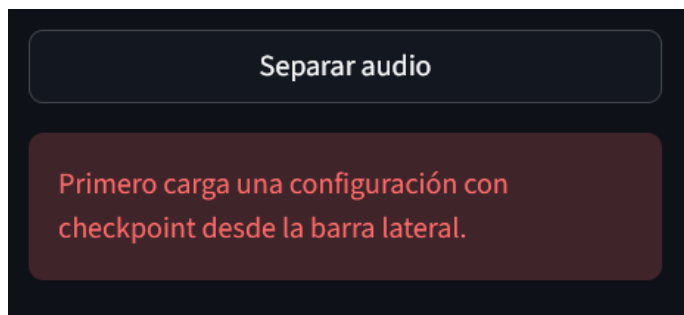


Figura 7.17: Ejemplo de mensaje de error mostrado en la interfaz al intentar separar una pista sin haber cargado una configuración de modelo previa.

Progreso: mientras se está realizando una acción que requiere de tiempo, como separar una pista, la interfaz muestra un indicador de espera. Ver figura.

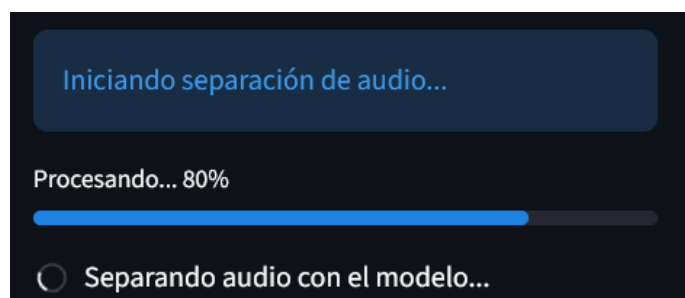


Figura 7.18: Ejemplo de indicador de espera en la interfaz mientras se está realizando el proceso de separar una pista.

Descarga de fuentes: por cada fuente separada el sistema permite al usuario guardar una copia en su equipo. Esto estaría pensado para una situación real en la que este sistema está *hosteado* en un servidor y no en ejecución local. Ver figura.

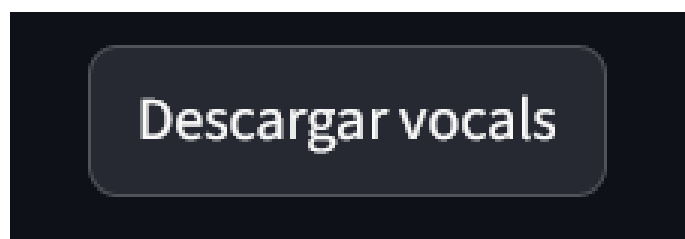


Figura 7.19: Botón de la interfaz que descarga un archivo wav de fuente en el almacenamiento del usuario.

Integración del sistema

CAPÍTULO 8



Backend

El sistema está conectado internamente a partir de módulos. Estos son entrenamiento, inferencia, despliegue, evaluación e interfaz. La conexión principal es el archivo ***config.py***, el cuál contiene un diccionario de configuración que se va modificando a lo largo de las diferentes rutas de ejecución que parten desde *main.py*. El flujo de ejecución y comunicación entre módulos sigue el diagrama a continuación:

8. INTEGRACIÓN DEL SISTEMA

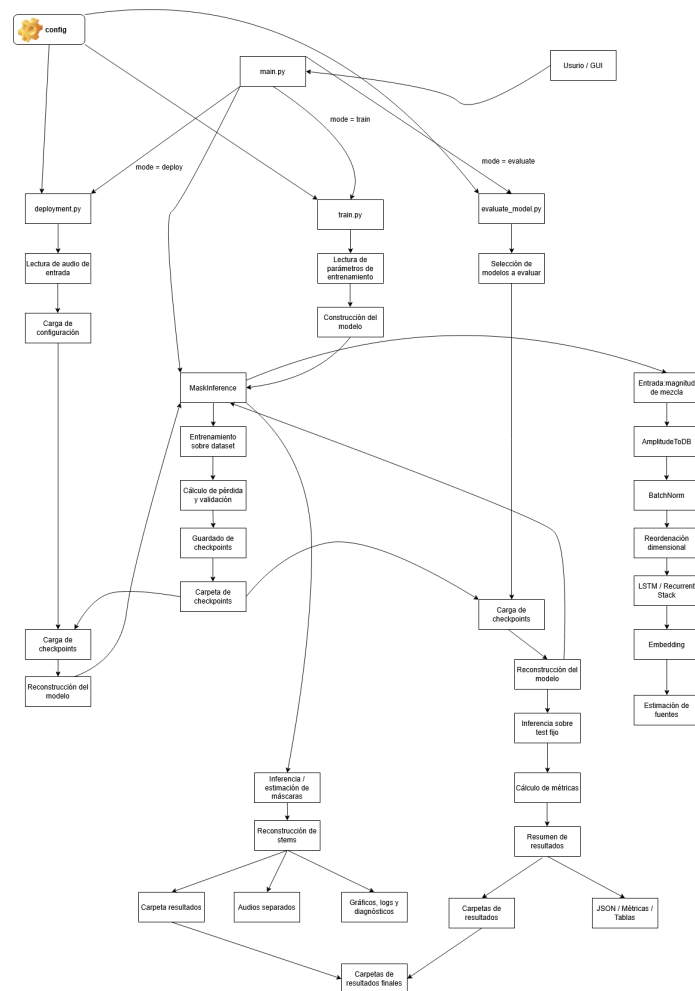


Figura 8.1: Diagrama de flujo para el backend

Y por módulos:

- **main.py** : punto de entrada principal, recibe por comando y ejecuta entrenamiento/despliegue/evaluación
- **config.py** : fuente central de parámetros, entre los que están parámetros de la STFT, hiperparámetros del modelo, función de pérdida, número de fuentes, rutas y dispositivo de ejecución.
- **train.py** : lee configuración, construye MaskInference, entrena, calcula pérdida y validación y guarda checkpoints en la carpeta checkpoints.
- **deployment.py** : infiere máscara sobre un audio concreto. Lee audio de entrada, carga el checkpoint, reconstruye el modelo, ejecuta inferencia, reconstruye stems y guarda resultados.

-
- `evaluate_models.py` : evalúa sistemáticamente modelos ya entrenados sobre el conjunto de test fijo. Selecciona modelos, carga checkpoints, reconstruye cada modelo, ejecuta una inferencia sobre el conjunto de test y calcula métricas y guarda resúmenes en `.json`.
 - `MaskInference` : contiene la arquitectura del modelo y la función de forward. Toma como entrada la magnitud de mezcla y aplica: Amplitud de ToDB, BatchNorm, reordenación dimensional, LSTM/RecurrentStack, Embedding, máscaras estimadas y estimación de fuentes.
 - Carpetas de checkpoints : guardan archivos `.pth` ordenados por función de pérdida y por número de fuentes.
 - Carpetas de resultados : guardan las salidas del backend. Contienen fuentes separados junto con mezcla original, métricas, gráficos y ficheros resumen.

Comunicación entre módulos

La comunicación entre módulos es la definida en el diagrama de flujo que se presenta en la figura 7.1. El núcleo principal de la configuración entre módulos son los archivos `main.py` y `config.py`. En el archivo `main` se decide la acción que se va a ejecutar y se llama a una función que lleva a cabo la tarea requerida. Previamente se cambian valores del archivo `config`, por ejemplo, la función de pérdida.

- 1. CLI / GUI modifica configuración
- 2. Se cargan datos o audio externo.
- 3. Se calcula STFT.
- 4. El modelo estima máscaras.
- 5. `run_inference` reconstruye audio.
- 6. `deployment` guarda stems.
- 7. Evaluación calcula métricas.
- 8. Resultados se guardan en carpeta de checkpoints/JSON/gráficas/tablas.

CAPÍTULO

Evaluación del sistema

9



9.1. Métricas utilizadas

En esta sección se explicará qué se ha medido, por qué, y las limitaciones encontradas.

- **SI-SDR:** Como métrica principal se ha utilizado **SI-SDR** ya que es una métrica habitual en separación de fuentes y permite comparar la señal estimada con la señal de referencia. Mide cuánta parte de la estimación se alinea con la fuente objetivo, frente a la parte que puede considerarse distorsión.

A la hora de la interpretación de los resultados, para SI-SDR, se valora a mejor un valor mas alto de esta métrica mientras que un valor mas bajo implica peor separación.

$$\text{SI-SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{noise}}\|^2}$$

- **Baseline de mezcla:** Un modelo no se considera útil si no supera el baseline de mezcla en el conjunto de test. Por ello, se ha añadido un baseline trivial, donde se utiliza la mezcla original como estimación de cada fuente para así detectar modelos que generan audio, pero no separan de verdad. Se utiliza como criterio mínimo para calificar una fuente como separación y si no se supera quiere decir que el modelo genera audio pero no información útil.

- **Métricas de agregación:**

Se han utilizado: media, da una medida de rendimiento promedio para un conjunto de valores de, por ejemplo SI-SDR. Mediana: da un valor de robustez frente a *outliers*. Desviación típica: determina la distancia a

la que están la mayoría de los valores de la media y se ha utilizado para valorar la estabilidad entre pistas.

■ **Métricas complementarias:**

Se han utilizado también varias métricas complementarias a modo de depuración y de diagnóstico: **correlación entre fuentes de salida y mezcla, estadísticas de máscara, matriz de correlación estimación-referencia**

9.2. Validación del pipeline de evaluación

La validez de los resultados del enfoque investigador del proyecto se basa en la comparabilidad de estos mismos, y , por ello, la validación de estos debería de responder a las siguientes preguntas:

- ¿Qué problema había antes?
- ¿Qué se hizo para validar la evaluación?
- ¿Por qué esto hace que los resultados sean comparables?
- ¿Qué se corrigió respecto a versiones anteriores?
- ¿Qué papel tiene el baseline?

Protocolo de evaluación utilizado:

El objetivo de crear un protocolo de evaluación no ha sido solo tener un proceso de ejecución sin errores, si no que también, y realmente de más importancia, asegurar que las métricas obtenidas reflejasen realmente las capacidades de cada función de pérdida en el modelo. El protocolo de evaluación propuesto es el siguiente:

- 1. Uso de *dataset* / *representative_test*.
- 2. Evaluación de todos los modelos sobre las mismas pistas.
- 3. Normalización de referencias y estimaciones a mono-canal.
- 4. Se recortan las pistas a una longitud común
- 5. Se usa la misma función de despliegue del modelo que en *deployment*.
- 6. Se guarda un sumario en formato *JSON* para cada modelo.

Esto se ha hecho siguiendo las recomendaciones habituales de reproducibilidad en *Machine Learning* [17]: documentar *dataset*, *splits*, preprocesado, hiperparámetros y resultados. La checklist de reproducibilidad incluye precisamente detalles del *dataset*, particiones *train/validation/test*, datos excluidos y pasos de preprocesado.

Inicialmente el proceso, aunque generase archivos, esto no significaba que los resultados fuesen realmente los buscados, en particular, porque las pistas

de salida eran exactamente iguales a las pistas de entrada en la mayoría de los casos, por lo que algo se estaba haciendo mal. Para dar alguna medida de validez a los resultados obtenidos, se trasladó el conjunto de test a un test fijo, se creó la validación de *baseline*, se añadió una sección en el código donde se normalizaban las pistas en mono-canal, recorte de pistas a una longitud común, impresión de métricas de análisis de las máscaras por pantalla en el entrenamiento y normalización del flujo de inferencia para todas las funciones de pérdida, donde previamente existían variaciones. A partir de aquí, todos los modelos pasan a evaluarse sobre las mismas pistas, con el mismo preprocesado, la misma métrica de evaluación y el mismo flujo de inferencia, además, añadir la métrica *baseline* ayuda a identificar los resultados que generaban audios pero no separaban mas allá de copiar la mezcla de entrada.

El protocolo de evaluación final queda definido así:

1. Cargar el conjunto de test fijo *representative_test*.
2. Evaluar todos los modelos sobre las mismas pistas.
3. Ejecutar la inferencia mediante la misma función utilizada en el despliegue.
4. Convertir referencias y estimaciones a mono-canal.
5. Recortar referencias y estimaciones a una longitud común.
6. Mantener un orden fijo de fuentes para dos y cuatro *stems*.
7. Calcular SI-SDR para cada modelo y pista.
8. Comparar los resultados frente al baseline de mezcla.
9. Guardar los resultados y sumarios de evaluación en archivos estructurados.

9.3. Resultados obtenidos

Una vez definido y validado el pipeline de evaluación, se aplicó el protocolo anterior a los modelos entrenados. Los resultados se presentan separando los escenarios de dos y cuatro fuentes, ya que ambos problemas presentan distinta dificultad y no son directamente comparables.

9. EVALUACIÓN DEL SISTEMA

Cuadro 9.1: Tabla para los resultados de separación en 2 fuentes

Función de pérdida	Grupo	Checkpoint	SI-SDRi medio	SI-SDRi mediano	Desviación típica	SI-SAR medio	Supera baseline
l1	principal	13	-7.157	-6.923	1.765	3.582	No
l2	principal	8	-2.515	-2.575	0.632	11.445	No
l1_freq	principal	13	-7.158	-6.924	1.762	3.578	No
l2_freq	principal	8	-2.262	-2.343	0.584	12.273	No
logl1	principal	69	-10.745	-11.134	2.771	-1.743	No
logl2	principal	77	-9.575	-9.118	2.675	0.012	No
log_mag	principal	56	-8.343	-8.071	2.066	2.130	No
log_compressed_l2	principal	45	-8.929	-8.540	2.198	0.500	No
lpsa	principal	61	—	—	—	—	—
mask_l1	principal	70	-8.239	-7.976	1.907	2.107	No
l_mrs	avanzado	—	—	—	—	—	—
lpsa_phase	avanzado	100	—	—	—	—	—
deep_feature	experimental	20	-0.091	-0.098	0.063	22.996	No
deep_feature_emd	experimental	21	-0.081	-0.082	0.042	23.435	No

Resultados en dos fuentes

Cuadro 9.2: Tabla para los resultados de separación en 4 fuentes

Función de pérdida	Grupo	Checkpoint	SI-SDRi medio	SI-SDRi mediano	Desviación típica	SI-SAR medio	Supera baseline
l1	principal	24	-4.811	-5.085	1.650	-1.933	No
l2	principal	4	-0.065	-0.066	0.038	20.263	No
l1_freq	principal	24	-4.814	-5.089	1.651	-1.940	No
l2_freq	principal	4	-0.069	-0.070	0.037	20.437	No
logl1	principal	65	-9.356	-9.176	2.587	-8.803	No
logl2	principal	4	-5.231	-5.183	1.850	-2.760	No
log_mag	principal	61	-6.767	-6.589	2.151	-4.835	No
log_compressed_l2	principal	65	-6.801	-6.708	1.946	-5.555	No
lpsa	principal	—	—	—	—	—	—
mask_l1	principal	33	-6.424	-6.297	2.025	-4.469	No
l_mrs	avanzado	—	—	—	—	—	—
lpsa_phase	avanzado	99	—	—	—	—	—
deep_feature	experimental	1	-0.018	-0.020	0.055	16.532	No
deep_feature_emd	experimental	3	0.006	0.004	0.059	17.562	Sí

Resultados en cuatro fuentes

9.3. Resultados obtenidos

Cuadro 9.3: Ranking para los resultados de separación en 2 fuentes

“ “

Rank	Función de pérdida	SI-SDR medio	SI-SDR mediano	SI-SDR STD	SI-SAR medio	Supera baseline
1	deep_feature_emd	-0.099	-0.109	0.063	23.435	No
2	deep_feature	-0.109	-0.104	0.091	22.996	No
3	l2_freq	-2.281	-2.349	0.563	12.273	No
4	l2	-2.533	-2.622	0.608	11.445	No
5	l1	-7.175	-6.975	1.737	3.582	No
6	l1_freq	-7.176	-6.976	1.734	3.578	No
7	lpsa_phase	-8.176	-8.721	1.915	–	No
8	mask_l1	-8.257	-8.027	1.871	2.107	No
9	log_mag	-8.361	-8.123	2.033	2.130	No
10	log_compressed_l2	-8.947	-8.556	2.164	0.500	No
11	lpsa	-9.423	-8.957	2.354	–	No
12	logl2	-9.593	-9.140	2.642	0.012	No
13	logl1	-10.763	-11.084	2.740	-1.743	No

“ “

Cuadro 9.4: Ranking para los resultados de separación en 4 fuentes

“ “

Rank	Función de pérdida	SI-SDR medio	SI-SDR mediano	SI-SDR STD	SI-SAR medio	Supera baseline
1	deep_feature_emd	-5.375	-5.260	0.440	17.562	Sí
2	deep_feature	-5.399	-5.320	0.413	16.532	No
3	l2	-5.445	-5.405	0.428	20.263	No
4	l2_freq	-5.449	-5.396	0.425	20.437	No
5	lpsa_phase	-6.423	-6.491	0.862	–	No
6	l1	-10.191	-10.449	1.727	-1.933	No
7	l1_freq	-10.194	-10.452	1.727	-1.940	No
8	logl2	-10.611	-10.651	1.880	-2.760	No
9	mask_l1	-11.805	-11.594	2.076	-4.469	No
10	log_mag	-12.147	-11.922	2.202	-4.835	No
11	log_compressed_l2	-12.181	-12.030	2.004	-5.555	No
12	logl1	-14.737	-14.475	2.562	-8.803	No

“ “

Rankings

Graficos de barras Se han llevado a cabo dos gráficos de barras para dar una imagen que represente visualmente los resultados:

9. EVALUACIÓN DEL SISTEMA

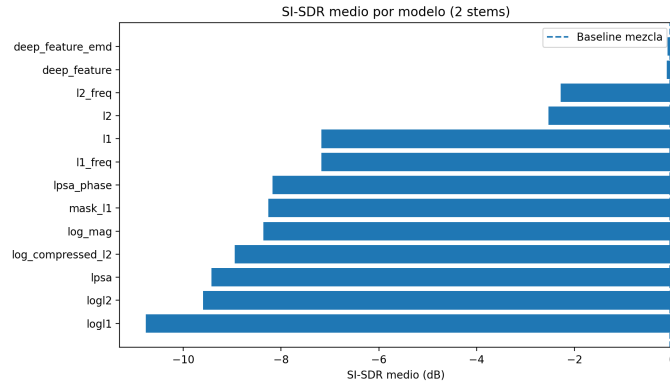


Figura 9.1: Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de dos fuentes

[5]

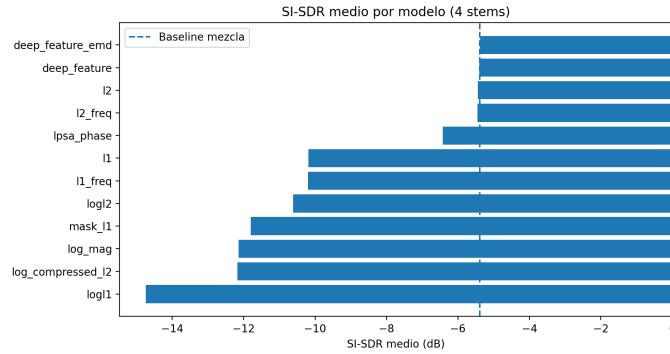


Figura 9.2: Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de cuatro fuentes

[5]

Discusión y análisis

Limitaciones y margen de mejora

A lo largo del desarrollo del trabajo, se ha tomado consciencia de las limitaciones que se pueden presentar en este tipo de tarea y que resultan en un peor desempeño del sistema.

- Fase de mezcla: como ya se ha comentado previamente, el uso de la fase de mezcla como estimación de la fase de las fuentes para reconstrucción limita en gran parte la tarea del modelo. Durante el entrenamiento y la

inferencia, la red aprende a estimar máscaras aplicadas sobre la magnitud de la STFT de la mezcla, y posteriormente, se reutiliza la fase de la mezcla como aproximación para reconstruir la señal temporal de cada fuente.

Este procedimiento simplifica considerablemente el problema, ya que evita tener que modelar la componente de fase pero también introduce una limitación importante. La fase real de cada fuente, como ya se comenta previamente en la memoria, no tiene por qué coincidir con la fase de la mezcla por lo que esto puede resultar en la presencia de ruido o pérdida de calidad perceptual, especialmente en casos donde varias fuentes se solapan [5].

- **Arquitectura simple:** la arquitectura utilizada se basa en un modelo de inferencia de máscaras con una red recurrente LSTM. Esta selección permite trabajar con una estructura interpretable y adecuada para el procesamiento de espectrogramas como secuencias temporales y sin embargo se trata de una arquitectura sencilla en comparación con modelos actuales de separación musical, que suelen emplear redes más profundas, arquitecturas convolucionales o Transformers.
- **Entrenamiento en mono:** otra limitación relevante es que el sistema trabaja principalmente con señales mono-canal. Si bien esto reduce complejidad y facilita el entrenamiento, también implica la pérdida de información espacial de las señales estéreo. En música real, la presencia de instrumentos en los canales puede facilitar información útil a la hora de separar fuentes.

Si bien el procesamiento en mono-canal permite simplificar el problema y mantener un coste computacional asumible, también limita la capacidad del sistema para aprender sobre pistas especiales que podrían mejorar la calidad de la separación.

- **Recursos computacionales:** el entrenamiento de modelos de separación musical, y en general de modelos que trabajan con pistas de audio, requiere una cantidad considerable de recursos computacionales, especialmente cuando se trabaja con más de dos fuentes o funciones de pérdida complejas, como *l_mrs* o *lpsa_phase*.

Los recursos disponibles se limitaban a un procesador Intel de onceava generación y una tarjeta gráfica Nvidia RTX 3060Ti, que si bien no son recursos flojos, se quedan cortos para la tarea esperada, algunos entrenamientos llegaron a tardar tres días y el tamaño del *batch* no era especialmente grande. Estas restricciones han condicionado decisiones como el tamaño del modelo, duración de fragmentos de audio, número de iteraciones de entrenamiento o cantidad de configuraciones evaluadas.

- Limitaciones del dataset: el rendimiento del sistema también lo condiciona el *dataset* utilizado. Si bien MUSDB18 es una referencia habitual, su tamaño es limitado en comparación con los utilizados para el entrenamiento de sistemas comerciales o modelos a gran escala.

Además, el dataset no cubre en su totalidad toda la variedad posible de géneros, mezclas, estilos de producción o calidades de grabación y instrumentación. Esto afecta a la capacidad de generalización del modelo, principalmente. Si bien es complicado, porque en música, como en cualquier arte, no hay una norma fija que defina el objeto "música", la solución sería tener cantidad de datos grande por género, por ejemplo.

- Métricas objetivas y percepción auditiva: la evaluación del sistema se ha basado principalmente en métricas objetivas como SI-SDR que permiten comparar de forma cuantitativa las estimaciones generadas por los diferentes modelos. Se eligió utilizar estas métricas ya que se necesitaba una medida reproducible objetiva y homogénea para evaluar el rendimiento.

No obstante, las métricas objetivas no siempre reflejan completamente la percepción auditiva humana y de hecho, se repite en el estado del arte que las métricas numéricas no suelen ser una medida fiable de calidad y que es mejor conducirse por una opinión humana. Dos separaciones con valores similares de SI-SDR pueden percibirse de manera diferente en términos de ruido, naturalidad o inteligibilidad de la voz. Por ello, la escucha perceptual se consideró necesaria para interpretar los resultados. Se tenía intención de realizar una encuesta de evaluación pública con diferentes salidas del modelo para cada una de las funciones de pérdida, pero por falta de tiempo se terminó por dejar de lado.

- Interfaz demostrativa: la interfaz gráfica tiene un objetivo principalmente demostrativo. Su finalidad es dar facilidad para el uso del sistema a un usuario ajeno al entorno de un IDE.

Aunque la interfaz permite la interacción directa con el pipeline y visualizar gráficos y logs, no debe entenderse como un producto comercial final, ya que no se han contemplado aspectos como control de errores, optimización de tiempos de respuesta, *hosting*, diseño visual de calidad o gestión de usuarios.

En conjunto, estas limitaciones no invalidan el sistema desarrollado, si no que más bien delimitan su alcance. El proyecto se centra en construir y validar un flujo completo de entrenamiento, inferencia y evaluación, sumándole a esto la interfaz. Se deja como trabajo futuro la incorporación de arquitecturas más avanzadas, datasets variados, entrenamiento estéreo y evaluaciones más valorables.

CAPÍTULO

Conclusiones y trabajo futuro

10



Síntesis del trabajo realizado

Trabajo futuro

El sistema desarrollado ha cumplido con el objetivo de implementar un flujo completo de separación de fuentes musicales, incluyendo entrenamiento, inferencia, evaluación e interfaz gráfica. De todas formas, existen aun así diferentes líneas de mejora que permitirán ampliar el alcance del trabajo.

Inicialmente, una primera fase de trabajo futuro sería realizar una comparación directa en desempeño frente a modelos ampliamente utilizados en separación musical de la función de pérdida que mas desempeño presenta en este proyecto. En este trabajo dichos sistemas se han usado únicamente como referencia dentro del estado del arte, pero no se han tenido en cuenta para realización de una evaluación directa experimental bajo el mismo protocolo. Una comparación justa requeriría ejecutar todos los modelos sobre el mismo conjunto de test, con las mismas métricas y condiciones de evaluación, permitiendo así contextualizar de forma más precisa el rendimiento del sistema desarrollado.

Otra mejora considerable sería ampliar el conjunto de datos empleado. Si bien MUSDB18 presenta un conjunto de datos de referencia en separación de fuentes, su tamaño es limitado frente a colecciones más amplias utilizadas en, por ejemplo, competencias como MDX.

Por otro lado, sería también un punto a favor estudiar la implementación de estructuras de arquitectura híbridas que combinen información en el dominio temporal con información en el dominio tiempo-frecuencia. El sistema trabaja con magnitudes de la STFT y aplica la fase de la mezcla sobre las magnitudes de las fuentes para reconstruir las fuentes. Los modelos híbridos aprovechan la interpretabilidad de las representaciones espectrales y la capacidad de los modelos en forma de onda para aprender información temporal y de fase de forma más directa.

Otra mejora posible sería adaptar el sistema para la separación en tiempo real. La implementación actual está orientada a procesar archivos de audio completos o fragmentos previamente cargados. Para trabajar a tiempo real sería necesario rediseñar el flujo de inferencia, añadir procesado de audio por bloques y optimizar la carga del modelo y controlar solapamiento entre ventanas.

También se podría añadir al sistema la posibilidad de separar un mayor número de fuentes. En este trabajo se han reducido las posibilidades por limitaciones a dos y cuatro fuentes, pero existen pistas donde sería relevante separar instrumentos más específicos como guitarra, piano, sintetizadores coros u otros elementos de la mezcla. Esta ampliación requeriría bases de datos con anotaciones de cinco fuentes, mayor capacidad de cómputo, modelos con mayor capacidad y una evaluación adaptada.

Finalmente, también se podría introducir la posibilidad de añadir efectos a las pistas de entrada para mejorar generalizaciones. La música contemporánea por lo general presenta una gran cantidad de post producción y cada caso es un mundo, se introducen muchos elementos que varían la producción inicial y esto varía en cada pista. Se planteaba inicialmente añadir esta posibilidad, pero por falta de tiempo y limitaciones no se pudo hacer. Entre las transformaciones a aplicar:

- Reverberación
- Shiftteo de pitch
- Ecualización
- Ruido

Finalmente no se utilizó por problemas entre versiones de librerías. Se menciona aquí una visión positiva de esta complicación, pues así se permite mantener una comparación controlada entre funciones de pérdida.

Código relevante

El anexo de código recoge fragmentos representativos del sistema. No se incluirán archivos completos, sino secciones seleccionadas para dar luz sobre las partes más importantes del proceso

- **main.py:** punto de entrada principal del *backend*. Permite seleccionar modo de ejecución, y su relevancia reside en centralizar el enrutamiento hacia los diferentes módulos del proyecto.
- **MaskInference.forward():** se incluye para documentar el proceso de entrenamiento y validación. Contiene la construcción del modelo, carga de atos, cálculo del valor de la pérdida, control de entrenamiento, *early_stopping* y guardado de *checkpoints*.
- **metrics.py:** contiene las funciones de pérdida utilizadas. Es relevante para entender cómo se comparan las estimaciones del modelo con las referencias y cómo se implementan las variantes de pérdidas utilizadas.
- **commons/inference.py** proceso de inferencia completo.
- **commons/audio_utils.py:** se incluye por su papel en la carga de modelos e inferencia de parámetros. Resultó especialmente importante para resolver incompatibilidades entre configuración actual y modelos previos.
- **deployment.py:** módulo encargado de aplicar el sistema a audios. Permite cargar un archivo de entrada, ejecutar separación, reconstruir fuentes y guardar resultados.
- **evaluate_mixture_baseline.py:** identifica baseline utilizado.
- **GUI/gui.py:** contiene la implementación de la interfaz gráfica. Integra la implementación de la interfaz gráfica para selección de moelo, carga de

checkpoints, separación de audio, visualización de resultados, generación de gráficas, consola y descarga de resultados.

Configuraciones

A continuación se recogen ejemplos de comandos utilizados para ejecutar las funciones del sistema. Las rutas y nombres concretos se deben adaptar en función de la configuración local del proyecto.

Entrenamiento individual. Permite entrenar un modelo concreto a partir de la configuración seleccionada.

```
python main.py --mode train
```

Entrenamiento por lotes. Permite lanzar varios entrenamientos asociados a distintas funciones de pérdida o configuraciones.

```
python -m scripts.train_V2
```

Evaluación de modelos. Ejecuta la evaluación de los modelos entrenados sobre el conjunto de test fijo.

```
python -m scripts.evaluate_models
```

Evaluación del baseline de mezcla. Da el valor baseline trivial que se utiliza para determinar si un modelo entrenado mejora una solución mínima.

```
python -m scripts.evaluate_mixture_baseline
```

Separación sobre audio externo. Aplica el proceso a un archivo de audio de entrada.

```
python main.py --mode deploy
```

Ejecución de la interfaz gráfica. Lanza en local la interfaz.

```
streamlit run GUI/gui.py
```

Generación del conjunto de test representativo. Genera una copia local de un subconjunto representativo de MUSDB18.

```
python -m scripts.script_musdb
```

Diagnóstico de salidas separadas. Compara las fuentes estimadas con las referencias mediante una matriz de correlación.

```
python -m scripts.diagnose_track_outputs
```

Inspección de checkpoints. Permite cargar un *checkpoint* y mostrar la forma de algunas capas relevantes del modelo.

```
python -m scripts.test
```

Generación de medias, medianas y desviaciones. Permite obtener en tres archivos diferentes los valores de la media, mediana y desviación típica de la SI-SDR de cada modelo.

```
python -m scripts.sacar_medianas
```

Resultados adicionales

Bibliografía

- [1] Society of Broadcast Engineers. “The Physical Nature of Sound”. En: *SBE Handbook Fundamentals of Audio* (2025). Accedido: 28 de agosto de 2025.
- [2] LibreTexts Physics. “Introduction to Sound”. En: *LibreTexts Physics* (2025). Accedido: 28 de agosto de 2025.
- [3] OpenStax. “Sound Intensity and Sound Level”. En: *OpenStax Physics* (2025). Accedido: 28 de agosto de 2025.
- [4] Source Separation Tutorial. *Phase in source separation*. <https://source-separation.github.io/tutorial/basics/phase.html>. Accedido: 28 de agosto de 2025. 2025.
- [5] Z. Rafii et al. “An Overview of Lead and Accompaniment Separation in Music”. En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (ago. de 2018), págs. 1307-1335.
- [6] HuggingFace. “AudioCourse”. En: (2023).
- [7] Zafar Rafii et al. “An Overview of Lead and Accompaniment Separation in Music”. En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (2018), págs. 1307-1335.
- [8] Fabian-Robert Stöter et al. “Open-Unmix: A Reference Implementation for Music Source Separation”. En: *Journal of Open Source Software* 4.41 (2019), pág. 1667.
- [9] Alexandre Défossez et al. “Demucs: Deep Extractor for Music Sources with extra unlabeled data remixed”. En: *arXiv preprint arXiv:1909.01174* (2019).
- [10] Romain Hennequin et al. “Spleeter: a fast and efficient music source separation tool with pre-trained models”. En: *Journal of Open Source Software* 5.50 (2020), pág. 2154.

- [11] Yashvi Chandola et al. *Deep Learning for Chest Radiographs: Computer-Aided Classification*. Academic Press, 2021.
- [12] Saarth Vardhan et al. “An Ensemble Approach to Music Source Separation: A Comparative Analysis of Conventional and Hierarchical Stem Separation”. En: *Arxiv* ().
- [13] Bing Yang et al. “Supervised Direct-Path Relative Transfer Function Learning for Binaural Sound Source Localization”. En: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2021.
- [14] Abhimanyu Sahai, Romann Weber y Brian McWilliams. “Spectrogram Feature Losses for Music Source Separation”. En: *European Signal Processing Conference (EUSIPCO)*. 2019.
- [15] Shasha Xia, Hao Li y Xueliang Zhang. “Using Optimal Ratio Mask as Training Target for Supervised Speech Separation”. En: (2017).
- [16] DeLiang Wang. “Time-Frequency Masking for Speech Separation and Its Potential for Hearing Aid Design”. En: *Trends in Amplification* (2008).
- [17] Jpineau. “Machine Learning Paper Reproducibility Checklist”. En: (2020).